多层感知机与卷积神经网络

授课教师：Hao Wang

AI Thrust，信息枢纽

Multi-layer perception and Convolutional Neural Networks

Instructor: Hao Wang

AI Thrust, information hub

上周回顾

Recaps of last week

线性分类器

Linear Classifier

一个更典型的框架

K 最近邻

```
def train(images, labels):
  # Machine learning!
  return model
```

记忆所有数据和标签

```
def predict(model, test_images):
  # Use model to predict labels
  return test_labels
```

预测最相似训练图像的标签

A more typical framework

K Nearest Neighbor

```
def train(images, labels):
  # Machine learning!
  return model
```

Memorize all data and labels

```
def predict(model, test_images):
  # Use model to predict labels
  return test_labels
```

Predict the label of the most similar training image

一个更典型的框架

线性分类器

```
def train(images, labels):
  # Machine learning!
  return model
```

定义一个模型 $f_W(x)$ 定义损失函数 L $f_W(x)$, y求解 W^*，使得 L 最小

```
def predict(model, test_images):
  # Use model to predict labels
  return test_labels
```

基于 $f_{W^*}(x)$预测标签

A more typical framework

Linear classifier

```
def train(images, labels):
  # Machine learning!
  return model
```

Define a model $f_W(x)$
Define a loss function $L(f_W(x), y)$
Solve W^* that leads to minimal L

```
def predict(model, test_images):
  # Use model to predict labels
  return test_labels
```

Predict the labels based on $f_{W^*}(x)$

正则化

L2 正则化： $e\ W\ (\quad W\ \#\ (\ \in_k W_k^2 L_1$ 正

则化： $e\ W\ (\quad W\ \%\ (\ \in_k W_k$ | |

1. “最大间隔”原则

在 SVM 中， 目标是找到一个能够分离各类别的决策边界 d

不使用正则化： 模型可能会过度努力地正确分类训练数据中的每一个噪声点，从而导致非常复杂的边界。

使用正则化： 最小化 W 2 在数学上等价于 **最大化间隔**。 更大的间隔通常会在新的、未见过的数据上带来更好的性能，因为它更具鲁棒性。

2. 控制“权重”

为什么需要小权重？ 如果某个权重 W_i 非常大，那么即使特定输入特征 x_i 发生极微小的变化，也会导致输出出现巨大波动。通过迫使权重保持较小，正则化可确保模型不会过度 a

依赖 训练集中的任何单一特征或特定噪声。这会带来一个**更平滑**的模型。

Regularization

L_2 regularization: $R(W) = \left|\left|W\right|\right|_2 = \sum_k W_k^2$

L_1 regularization: $R(W) = \left|\left|W\right|\right|_1 = \sum_k |W_k|$

1. The "Maximum Margin" Principle

In an SVM, the goal is to find a decision boundary that separates classes.

Without Regularization: The model might try too hard to correctly classify every single noise point in the training data, leading to a very complex boundary.

With Regularization: Minimizing $\left|\left|W\right|\right|_2$ is mathematically equivalent to **maximizing the margin.** A larger margin usually leads to better performance on new, unseen data because it’s more robust.

2. Controlling the "Weights"

Why small weights? If a weight W_i is very large, even a tiny change in a specific input feature x_i will cause a huge swing in the output. By forcing the weights to be small, regularization ensures that the model doesn't over-rely on any single feature or specific noise in the training set. This leads to a **smoother** model.

损失函数

比较两个概率分布：交叉熵

$$L = -\frac{1}{N}\sum_i y_i \log(q_i) = -\frac{1}{N}\sum_i \log\left(\frac{e^{s_{y_i}}}{\sum_j e^{s_j}}\right)$$



概率必须为 □□ =

概率之和必须为 1

cat	**3.2**	exp	**24.5**	归一化	**0.13**	比较 **1**
car	5.1		164.0		0.87	**0**
frog	-1.7		0.18		0.00	0

x □ $s(=_i, f\ )$ □⁻ $\frac{□^{-k}}{\sum_+ □^{-j}}$

Loss function

Comparing two probability distributions: Cross entropy

$$L = -\frac{1}{N}\sum_i p_i \log(q_i) = -\frac{1}{N}\sum_i \log\left(\frac{e^{s_{y_i}}}{\sum_j e^{s_j}}\right)$$



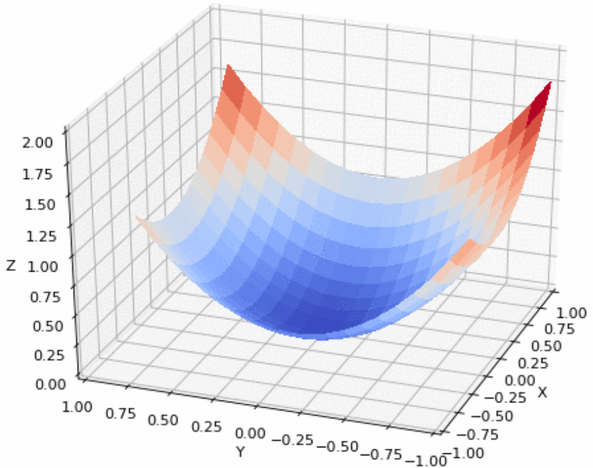
Probabilities must be >= 0

Probabilities must sum to 1

cat	**3.2**	exp	**24.5**	normalize	**0.13**	compare **1**
car	5.1		164.0		0.87	**0**
frog	-1.7		0.18		0.00	0

$s = f(x_i, W)$ e^s $\frac{e^{s_k}}{\sum_j e^{s_j}}$

梯度下降

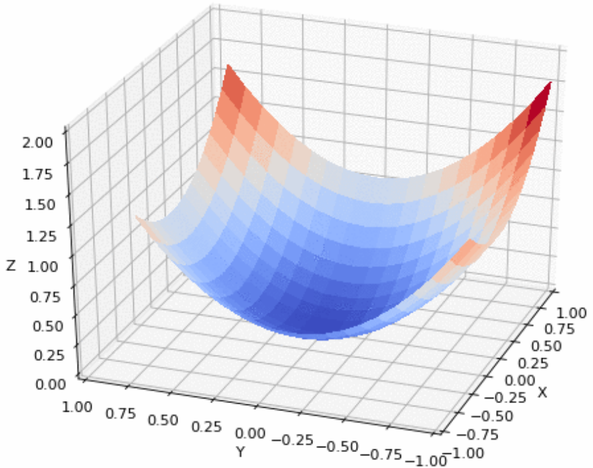


思路：沿着使损失下降最陡的方向前进。

$$W \; L \;\; W = \boldsymbol{x} \frac{jL}{jW}$$

步长　　梯度

Gradient descent



Idea: follow the direction that leads to steepest loss reduction.

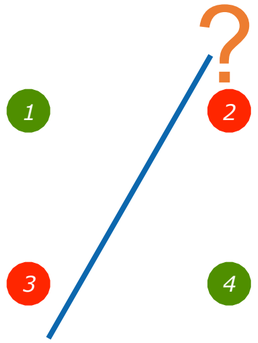
$$W \leftarrow W - \lambda \frac{\partial L}{\partial W}$$

Step size　　Gradient

本周：非线性模型

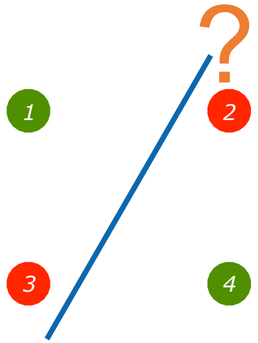
This week: Non-Linear Model

学习 XOR



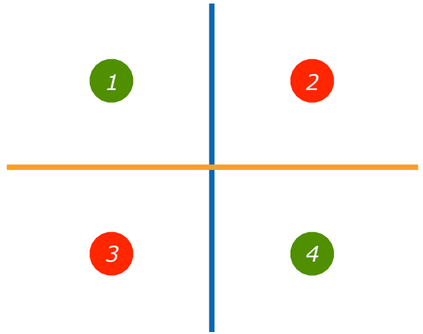
线性分类器无法处理 XOR 问题

Learning XOR

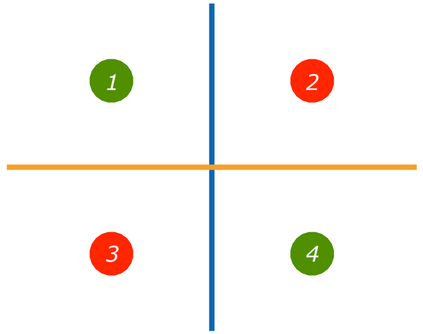


Linear classifier cannot handle XOR problem

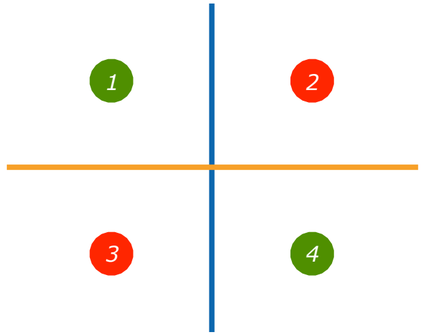
学习XOR



Learning XOR



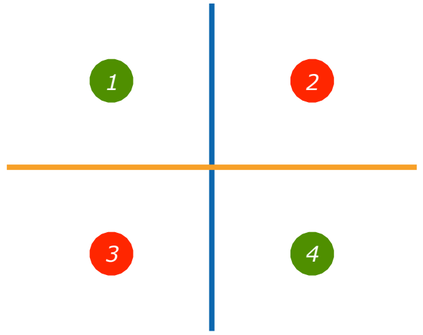
学习异或



	2	3	4	6
	+	-	+	-
	+	+	-	-
乘积	+	-	-	+

通过结合这两个线性特征，我们成功地将绿色点 (□) 与红色点 (-) 分离开来!

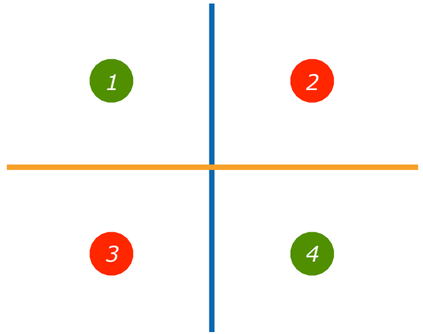
Learning XOR



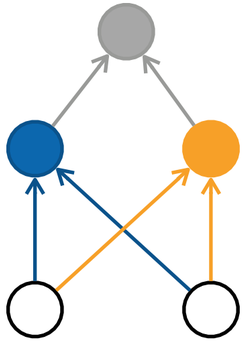
	1	2	3	4
	+	-	+	-
	+	+	-	-
product	+	-	-	+

By combining the two linear features, we have successfully separated the green points (+) from the red points (-)!

学习XOR



	2	3	4	6
	+	-	+	-
	+	+	-	-
乘积	+	-	-	+

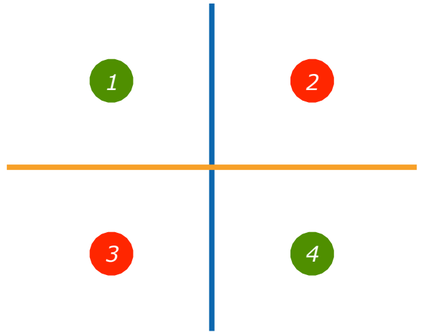


输出层： 该节点接收隐藏层的输出并将它们组合起来以进行最终分类（表格的最后几行）。

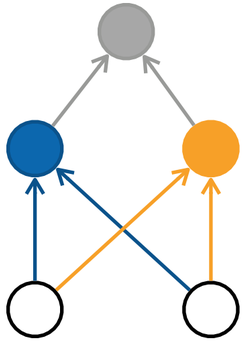
隐藏层：这些节点表示图中绘制的两条线。它们接收原始坐标，并应用线性变换（表的前两行）。

输入层：这些表示数据点的原始 x 和 y 坐标。

Learning XOR



	1	2	3	4
	+	-	+	-
	+	+	-	-
product	+	-	-	+

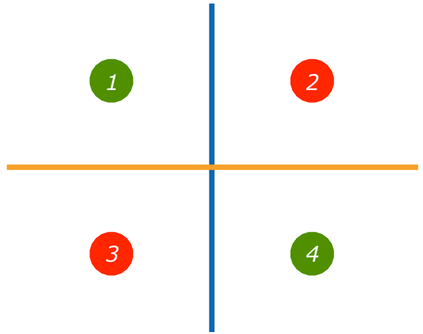


Output Layer: *This node takes the outputs from the hidden layer and combines them to make the final classification (the last rows of the table).*

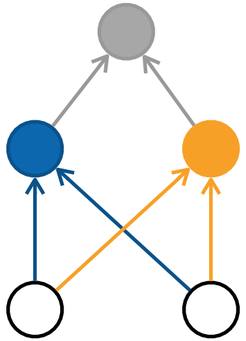
Hidden Layer: *These nodes represent the two lines drawn on the graph. They take the raw coordinates and apply the linear transformations (the first two rows of the table).*

Input Layer*: These represent the original x and y coordinates of the data points.*

学习异或



	2	3	4	6
	+	-	+	-
	+	+	-	-
乘积	+	-	-	+

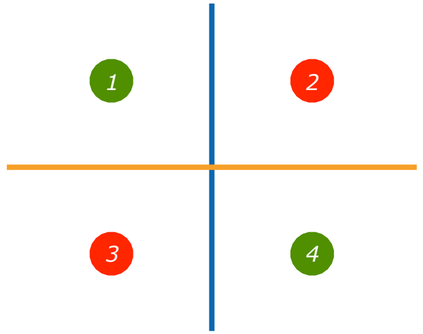


问题： 这是一个线性模型吗？为什么？

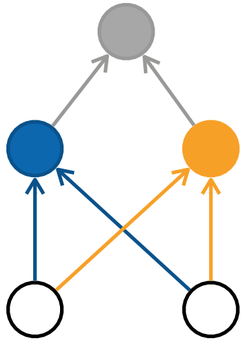
答案：不！

- **阈值**（激活函数，阶跃函数）
- **乘积运算**（将变量相乘，如 $x_1 \times x_2$）或应用逻辑规则是一种根本上的非线性运算。）

Learning XOR



	1	2	3	4
	+	-	+	-
	+	+	-	-
product	+	-	-	+

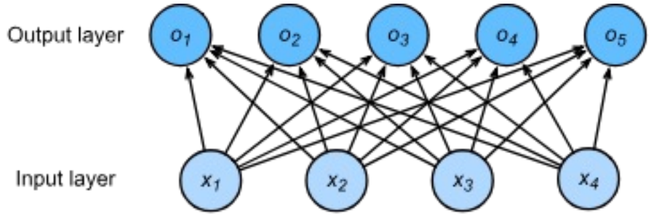


Question: *Is this a linear model? Why?*

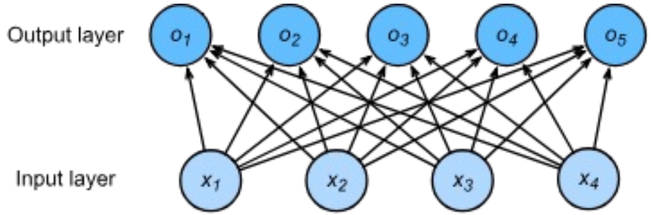
Answer: *No!*

- **The Threshold** *(The Activation Function, Step Function)*
- **The Product Operation** *(Multiplying variables together, like $x_1 \times x_2$) or applying logical rules is a fundamentally non-linear operation.)*

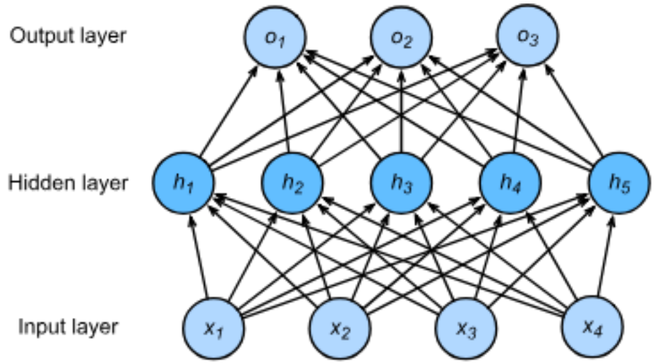
单隐藏层



Single Hidden Layer

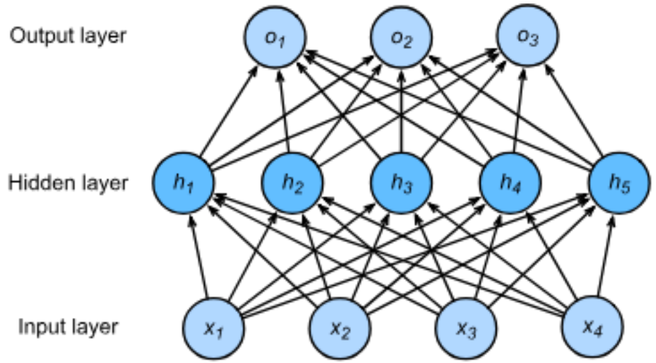


单隐藏层



超参数——隐藏层大小 m

Single Hidden Layer



Hyperparameter - size m of hidden layer

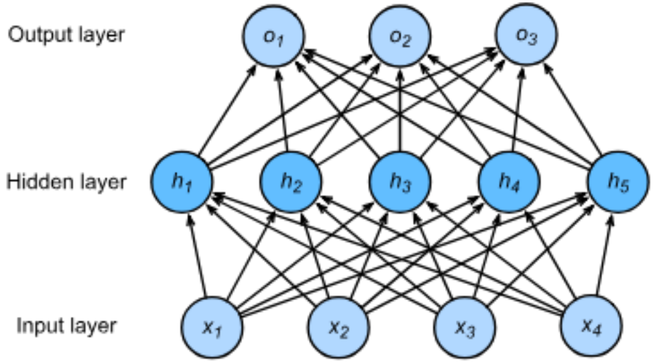
单隐藏层

- 输入 $\mathbf{x} \in \mathbb{R}^n$
- 隐藏 $\mathbf{W}_1 \in \mathbb{R}^{m \times n}$ $\mathbf{b}_1 \in \mathbb{R}^m$
- 输出 $\mathbf{W}_2 \in \mathbb{R}^m$, $\mathbf{b}_2 \in \mathbb{R}$

$$\mathbf{h} = \sigma(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1)$$

$$\mathbf{o} = \mathbf{w}^T\mathbf{h} + b_2$$

σ 是一种逐元素激活函数



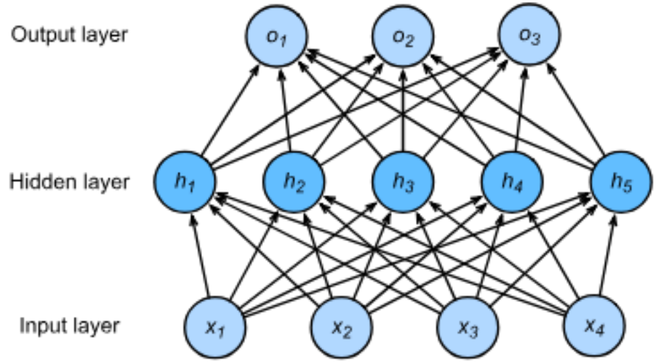
Single Hidden Layer

- Input $\mathbf{x} \in \mathbb{R}^n$
- Hidden $\mathbf{W}_1 \in \mathbb{R}^{m \times n}$, $\mathbf{b}_1 \in \mathbb{R}^m$
- Output $\mathbf{W}_2 \in \mathbb{R}^m$, $\mathbf{b}_2 \in \mathbb{R}$

$$\mathbf{h} = \sigma(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1)$$

$$\mathbf{o} = \mathbf{w}_2^T\mathbf{h} + b_2$$

σ is an element-wise activation function



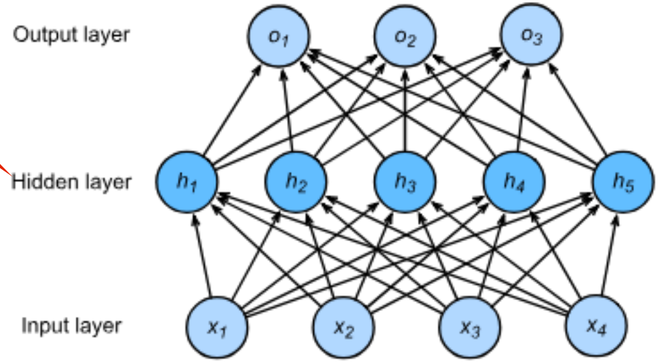
单隐藏层

为什么我们需要非线性激活?

$$\mathbf{h} = \sigma(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1)$$

$$\mathbf{o} = \mathbf{w}^T\mathbf{h} + b_2$$

σ 是一个逐元素的激活函数



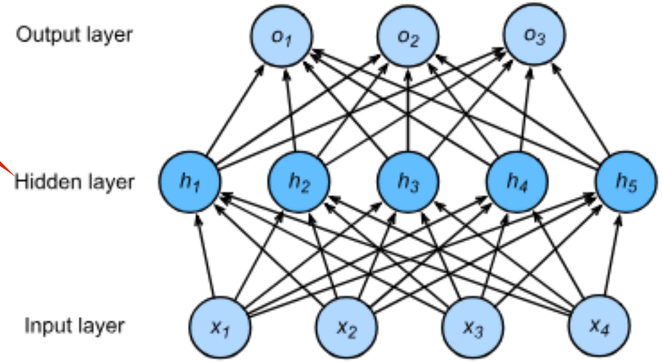
Single Hidden Layer

Why do we need a nonlinear activation?

$$\mathbf{h} = \sigma(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1)$$

$$\mathbf{o} = \mathbf{w}_2^T\mathbf{h} + b_2$$

σ is an element-wise activation function



单隐藏层

线性变换的线性变换是 仍然是线性变换!

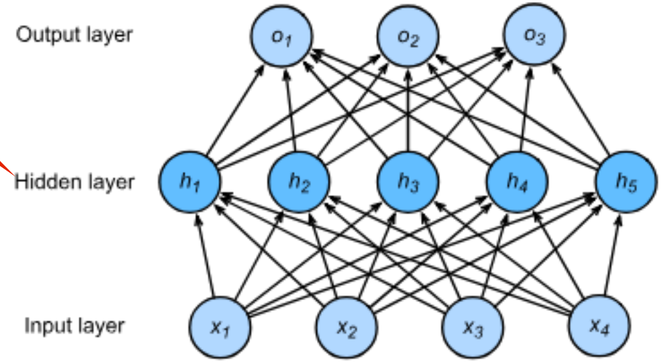
为什么我们需要非线性激活?

$$\mathbf{h} = \mathbf{W}_1\mathbf{x} + \mathbf{b}_1$$

$$\mathbf{o} = \mathbf{w}_2^T\mathbf{h} + b_2$$

因此 $o = \mathbf{w}^T\mathbf{W}_1\mathbf{x} + b'$

线性 ...



Single Hidden Layer

A linear transformation of a linear transformation is still a linear transformation!

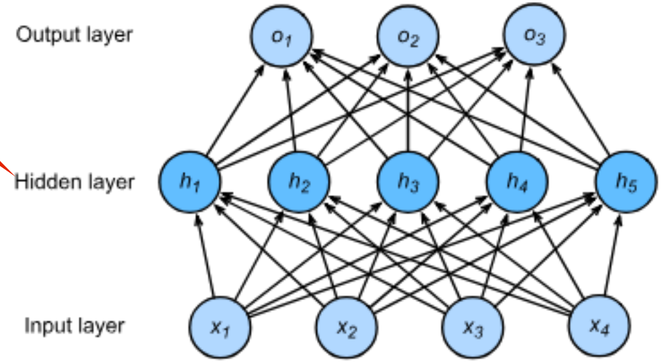
Why do we need a nonlinear activation?

$$\mathbf{h} = \mathbf{W}_1\mathbf{x} + \mathbf{b}_1$$

$$\mathbf{o} = \mathbf{w}_2^T\mathbf{h} + b_2$$

hence $o = \mathbf{w}_2^T\mathbf{W}_1\mathbf{x} + b'$

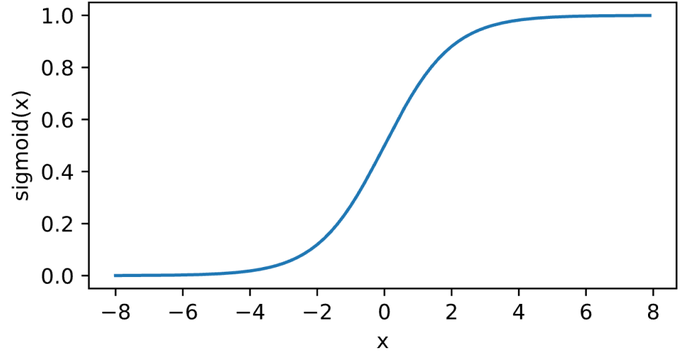
Linear …



Sigmoid 激活

将输入映射到 (0, 1)，一种平滑版本：$\sigma(x) = \begin{cases} 1 & \text{如果 } x > 0 \\ 0 & \text{否则} \end{cases}$

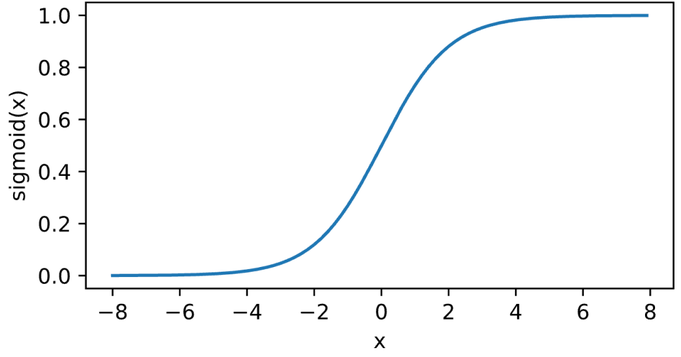
$\text{sigmoid}(x) = \frac{1}{1+\exp(-x)}$



Sigmoid Activation

Map input into (0, 1), a soft version of $\sigma(x) = \begin{cases} 1 & \text{if } x > 0 \\ 0 & \text{otherwise} \end{cases}$

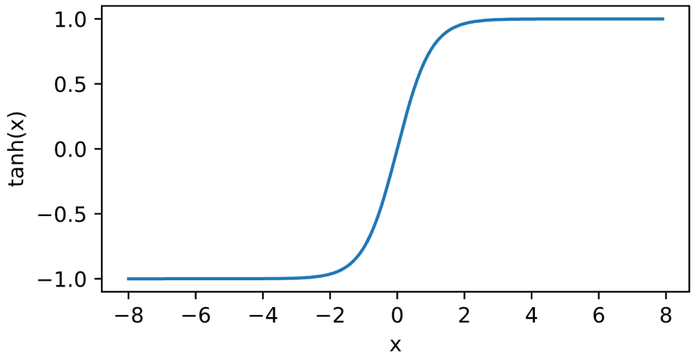
$\text{sigmoid}(x) = \frac{1}{1+\exp(-x)}$



Tanh 激活函数

将输入映射到 (-1, 1)

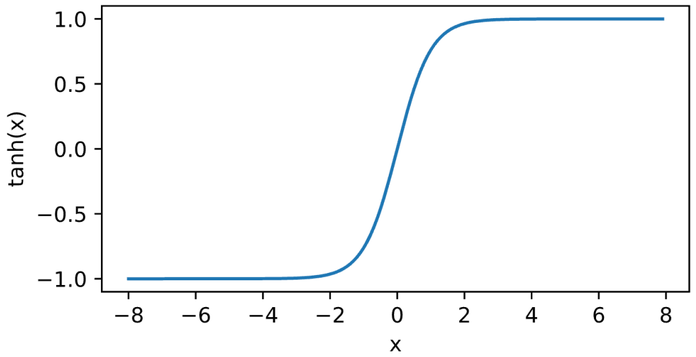
$$\text{anh}x = \frac{3 = \text{g0x}(=70)}{3 \times \text{g0x}(=70)}$$



Tanh Activation

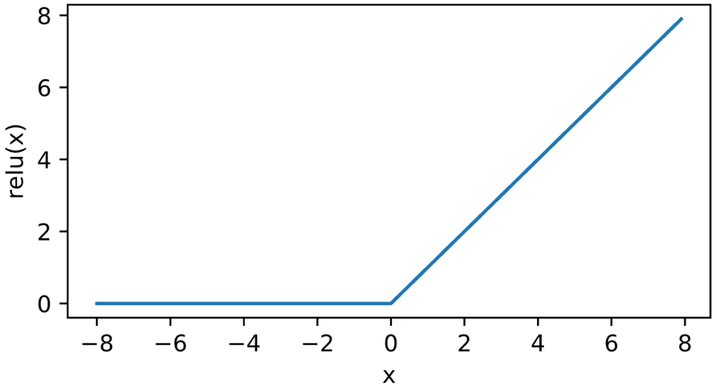
Map inputs into (-1, 1)

$$\tanh(x) = \frac{1 - \exp(-2x)}{1 + \exp(-2x)}$$



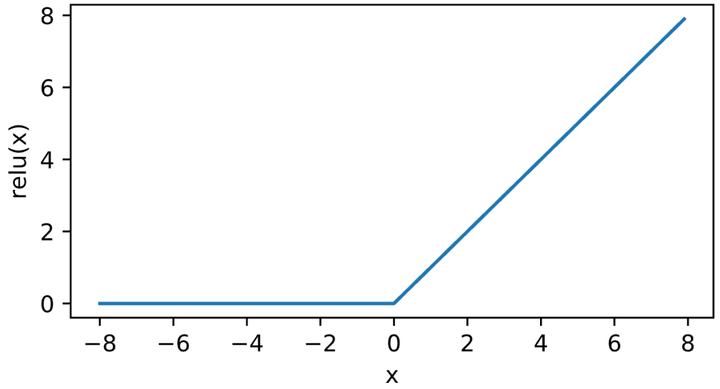
ReLU 激活

ReLU：修正线性单元：ReLU(*x*) = max(*x*,0)

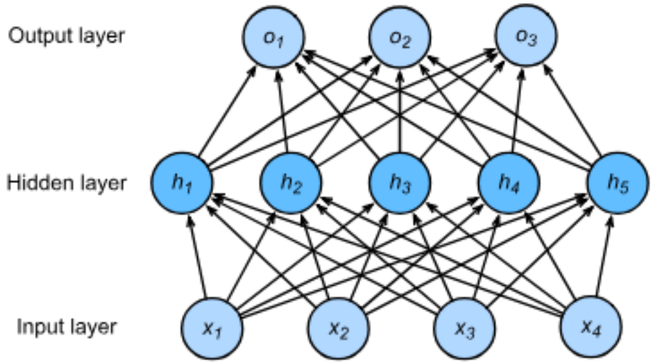


ReLU Activation

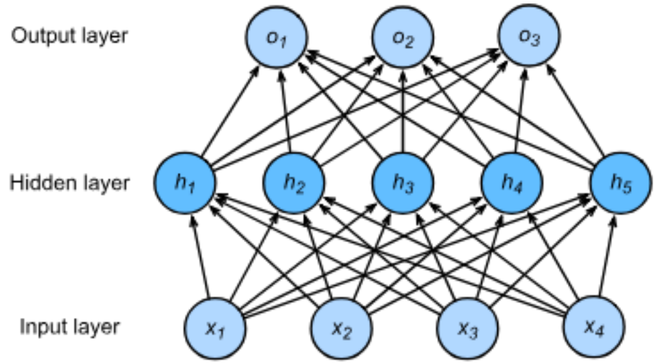
ReLU: rectified linear unit: ReLU(*x*) = max(*x*,0)



多类分类

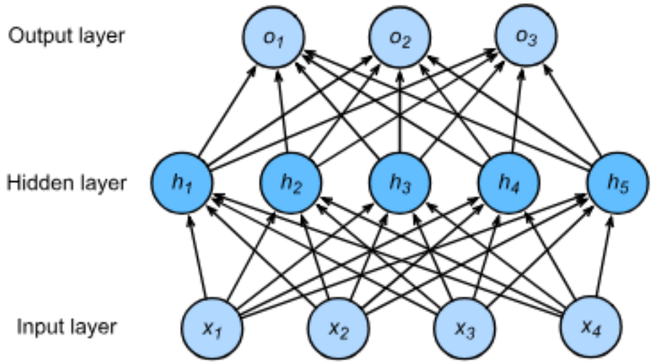


Multiclass Classification



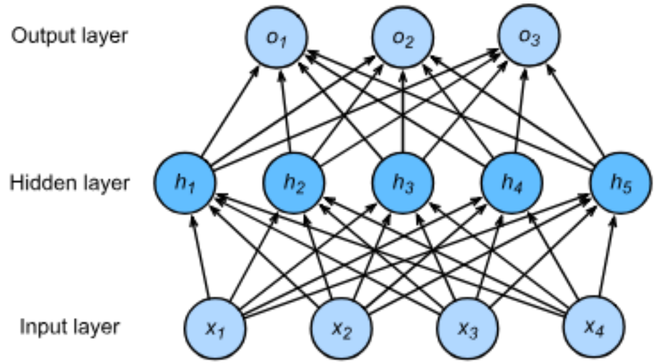
多类分类

$y_1 1 y_2 1 \ldots, y_k$? softmax($o_1 1 o_2 1 \ldots, o_k$)



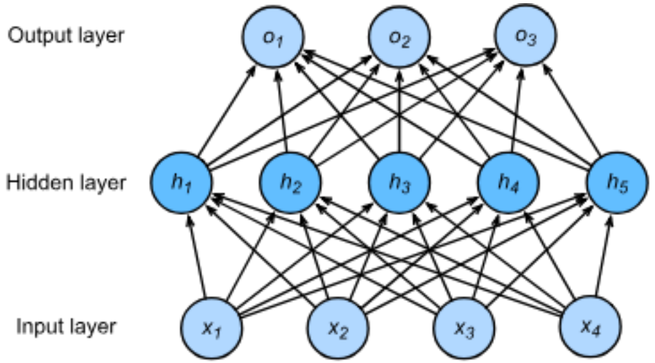
Multiclass Classification

$y_1, y_2, \ldots, y_k = \text{softmax}(o_1, o_2, \ldots, o_k)$



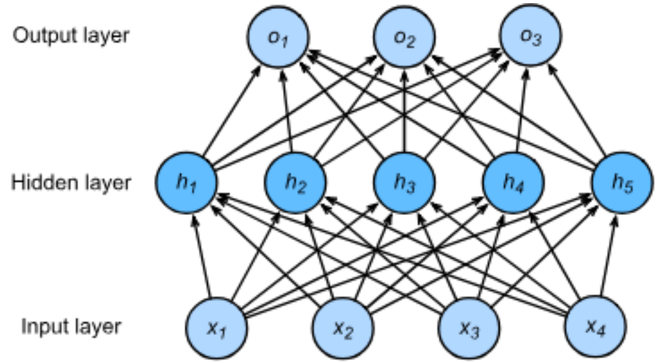
多类分类

- 输入 $\mathbf{x}\ e\mathrm{R}n$
- 隐藏 $\mathbf{W}_1$? $\mathrm{R}m\times n$ 和 $\mathbf{b}_1$? $\mathrm{R}m$
- 输出 $\mathbf{W}_2$? $\mathrm{R}m\times d$ 和$\mathbf{b}_2$? $\mathrm{R}d$



Multiclass Classification

- Input $\mathbf{x} \in \mathbb{R}^n$
- Hidden $\mathbf{W}_1 \in \mathbb{R}^{m\times n}$ and $\mathbf{b}_1 \in \mathbb{R}^m$
- Output $\mathbf{W}_2 \in \mathbb{R}^{m\times d}$ and $\mathbf{b}_2 \in \mathbb{R}^d$



多类分类

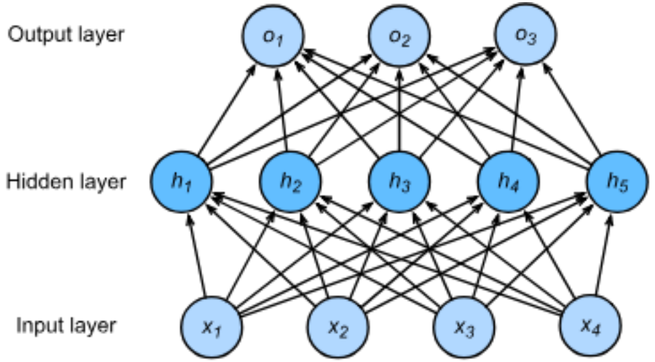
- 输入 $\mathbf{x} \in \mathbb{R}^n$
- 隐藏层 $\mathbf{W}_1 \in \mathbb{R}^{m \times n}$ 和 $\mathbf{b}_1 \in \mathbb{R}^m$
- 输出层 $\mathbf{W}_2 \in \mathbb{R}^{m \times d}$ 和 $\mathbf{b}_2 \in \mathbb{R}^d$

$$\mathbf{h} = \sigma(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1)$$

$$\mathbf{o} \in \mathbf{w}^T\mathbf{h} \in \mathbf{b}$$

$$_2 \quad ^2\mathbf{y} \in$$

$$\text{softmax}(o)$$



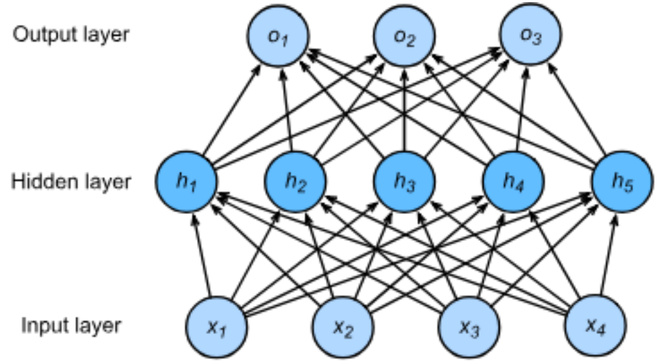
Multiclass Classification

- Input $\mathbf{x} \in \mathbb{R}^n$
- Hidden $\mathbf{W}_1 \in \mathbb{R}^{m \times n}$ and $\mathbf{b}_1 \in \mathbb{R}^m$
- Output $\mathbf{W}_2 \in \mathbb{R}^{m \times d}$ and $\mathbf{b}_2 \in \mathbb{R}^d$

$$\mathbf{h} = \sigma(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1)$$

$$\mathbf{o} = \mathbf{W}_2^T\mathbf{h} + \mathbf{b}_2$$

$$\mathbf{y} = \text{softmax}(o)$$

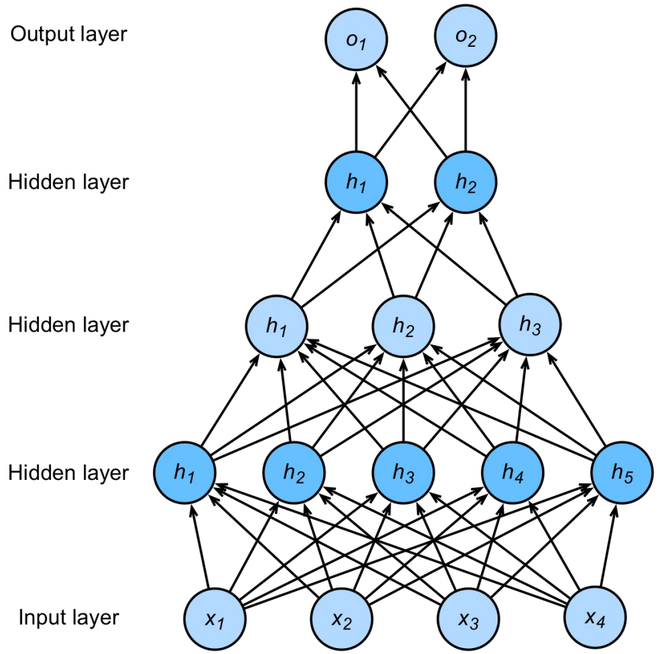


多个隐藏层

$$\mathbf{h}_1 = \sigma(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1)$$
$$\mathbf{h}_2 = \sigma(\mathbf{W}_2\mathbf{h}_1 + \mathbf{b}_2)$$
$$\mathbf{h}_3 = \sigma(\mathbf{W}_3\mathbf{h}_2 + \mathbf{b}_3)$$
$$\mathbf{o} = \mathbf{W}_4\mathbf{h}_3 + \mathbf{b}_4$$

超参数

- 隐藏层数量
- 每一层的隐藏层大小

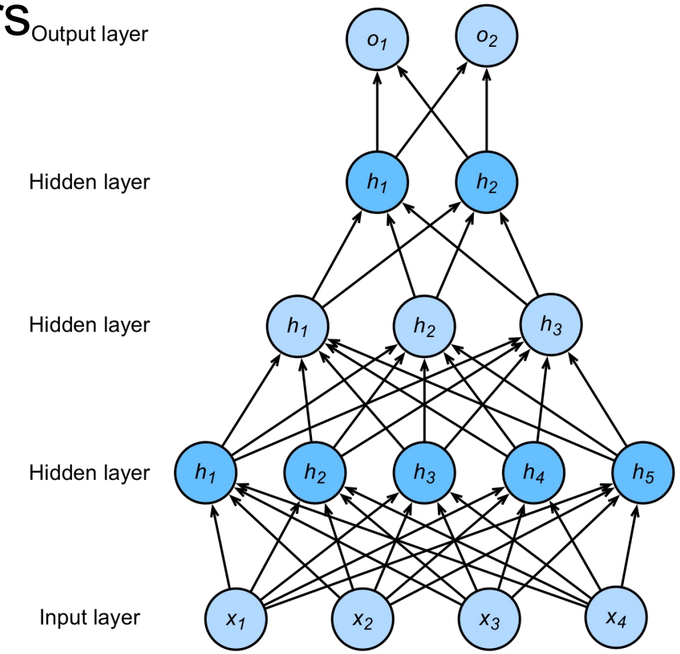


Multiple Hidden Layers

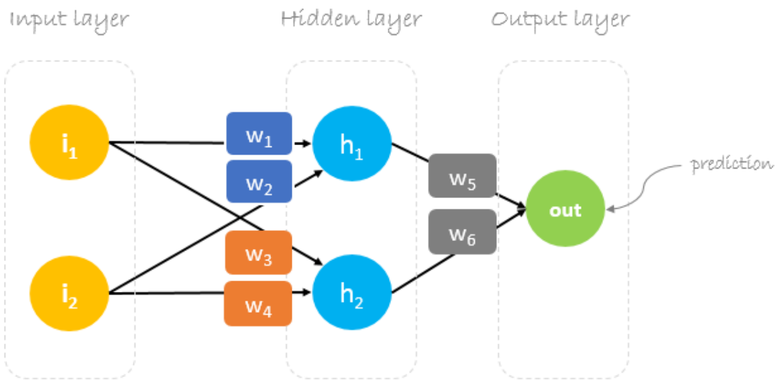
$$\mathbf{h}_1 = \sigma(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1)$$
$$\mathbf{h}_2 = \sigma(\mathbf{W}_2\mathbf{h}_1 + \mathbf{b}_2)$$
$$\mathbf{h}_3 = \sigma(\mathbf{W}_3\mathbf{h}_2 + \mathbf{b}_3)$$
$$\mathbf{o} = \mathbf{W}_4\mathbf{h}_3 + \mathbf{b}_4$$

Hyper-parameters

- Number of hidden layers
- Hidden size for each layer

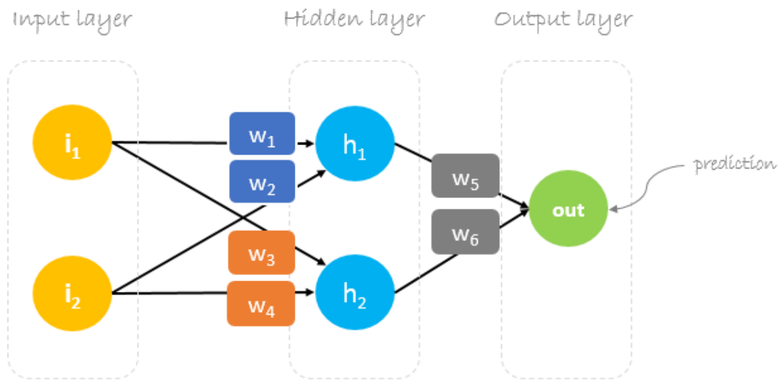


训练 MLP：反向传播



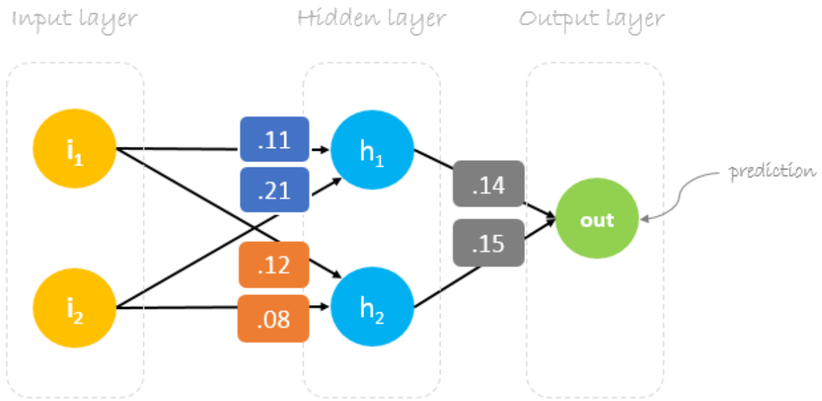
https://hmkcode.com/ai/backpropagation-step-by-step/

Training MLP: Backpropagation



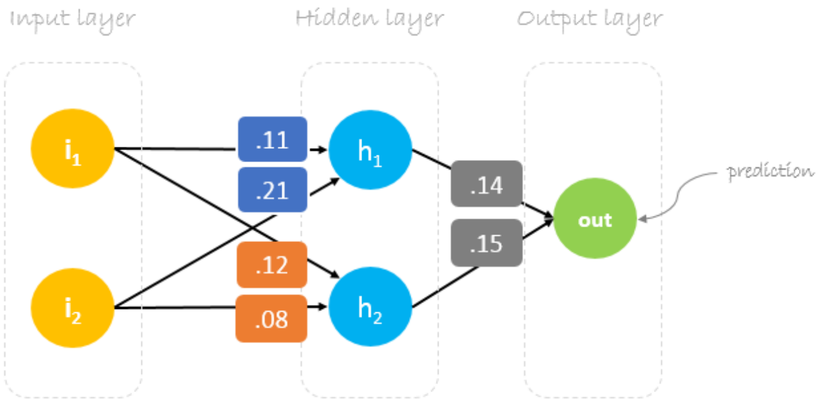
https://hmkcode.com/ai/backpropagation-step-by-step/

训练 MLP：反向传播



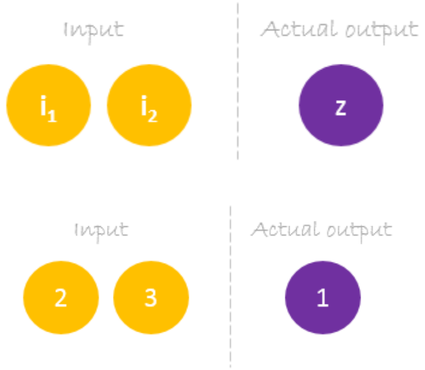
https://hmkcode.com/ai/backpropagation-step-by-step/

Training MLP: Backpropagation



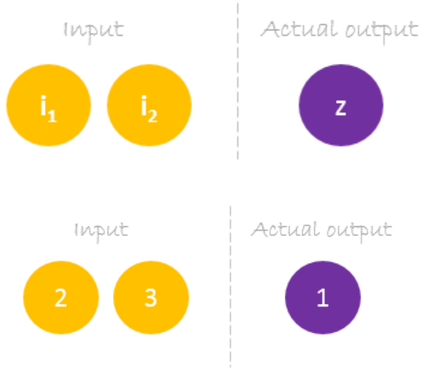
https://hmkcode.com/ai/backpropagation-step-by-step/

训练 MLP：反向传播



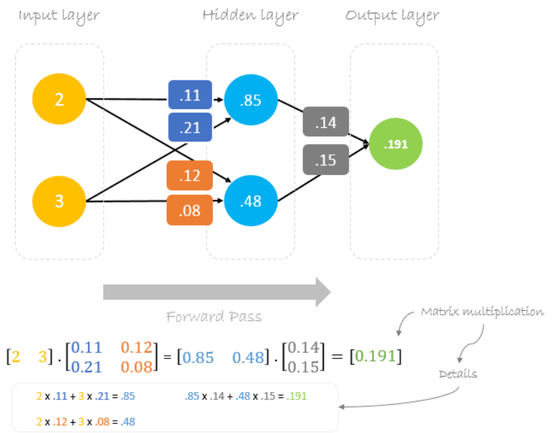
https://hmkcode.com/ai/backpropagation-step-by-step/

Training MLP: Backpropagation



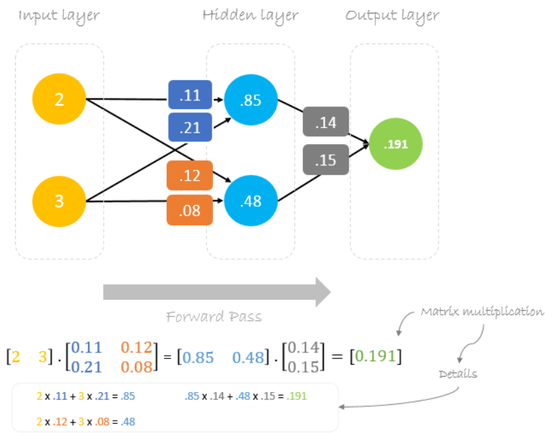
https://hmkcode.com/ai/backpropagation-step-by-step/

训练 MLP：反向传播



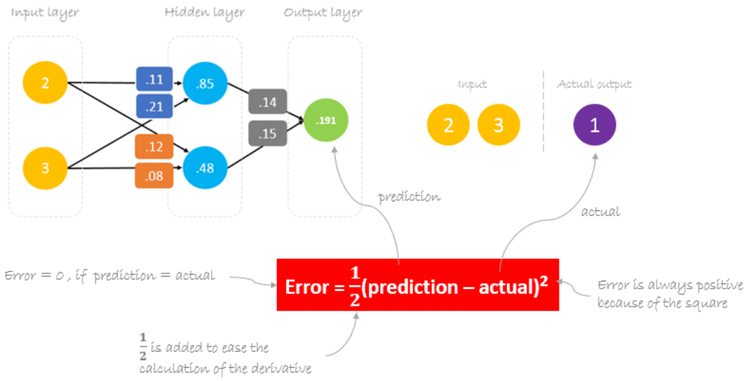
https://hmkcode.com/ai/backpropagation-step-by-step/

Training MLP: Backpropagation



https://hmkcode.com/ai/backpropagation-step-by-step/

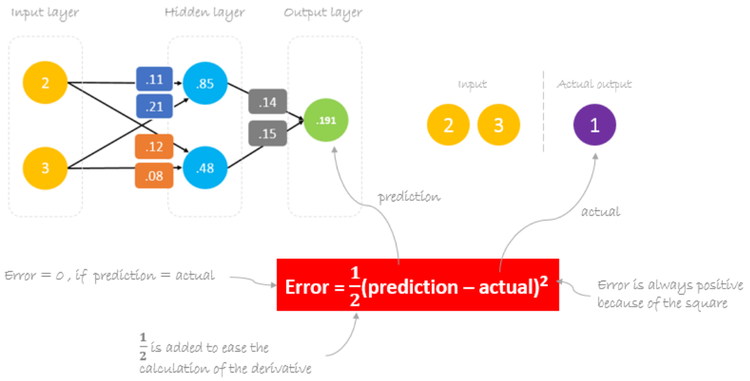
训练 MLP：反向传播



Error = $\frac{1}{2}(0.191 - 1.0)^2 = 0.327$

https://hmkcode.com/ai/backpropagation-step-by-step/

Training MLP: Backpropagation



Error = $\frac{1}{2}(0.191 - 1.0)^2 = 0.327$

https://hmkcode.com/ai/backpropagation-step-by-step/

训练 MLP：反向传播

$$\text{prediction} = \text{out}$$

$$\text{prediction} = (h_1)\, w_5 + (h_2)\, w_6$$

$$h_1 = i_1 w_1 + i_2 w_2$$
$$h_2 = i_1 w_3 + i_2 w_4$$

$$\text{prediction} = (i_1 w_1 + i_2 w_2)\, w_5 + (i_1 w_3 + i_2 w_4)\, w_6$$

to change ***prediction*** value,
we need to change ***weights***

https://hmkcode.com/ai/backpropagation-step-by-step/

Training MLP: Backpropagation

$$\text{prediction} = \text{out}$$

$$\text{prediction} = (h_1)\, w_5 + (h_2)\, w_6$$

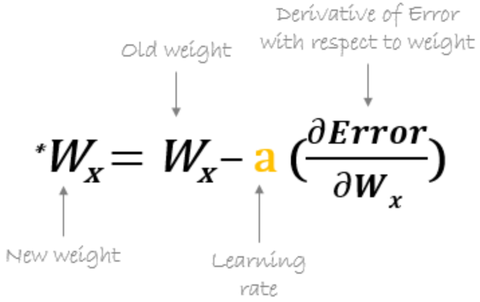
$$h_1 = i_1 w_1 + i_2 w_2$$
$$h_2 = i_1 w_3 + i_2 w_4$$

$$\text{prediction} = (i_1 w_1 + i_2 w_2)\, w_5 + (i_1 w_3 + i_2 w_4)\, w_6$$

to change ***prediction*** value,
we need to change ***weights***

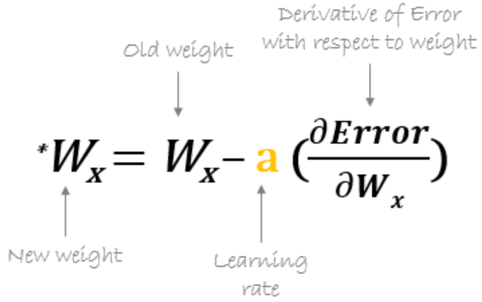
https://hmkcode.com/ai/backpropagation-step-by-step/

训练 MLP：反向传播



https://hmkcode.com/ai/backpropagation-step-by-step/

Training MLP: Backpropagation



https://hmkcode.com/ai/backpropagation-step-by-step/

训练 MLP：反向传播

$$^{*}W_6 = W_6 - \mathbf{a}\left(\frac{\partial Error}{\partial W_6}\right)$$

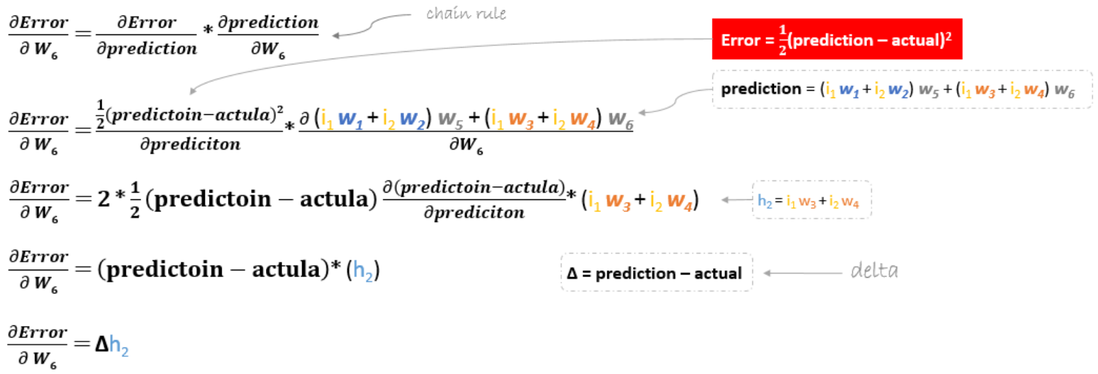
https://hmkcode.com/ai/backpropagation-step-by-step/

Training MLP: Backpropagation

$$^{*}W_6 = W_6 - \mathbf{a}\left(\frac{\partial Error}{\partial W_6}\right)$$

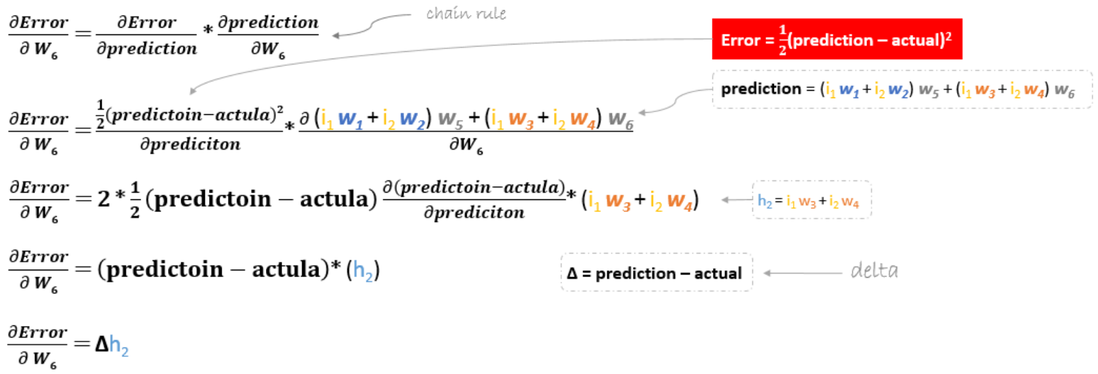
https://hmkcode.com/ai/backpropagation-step-by-step/

训练 MLP：反向传播



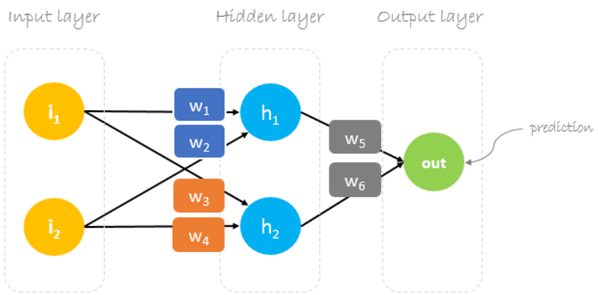
https://hmkcode.com/ai/backpropagation-step-by-step/

Training MLP: Backpropagation



https://hmkcode.com/ai/backpropagation-step-by-step/

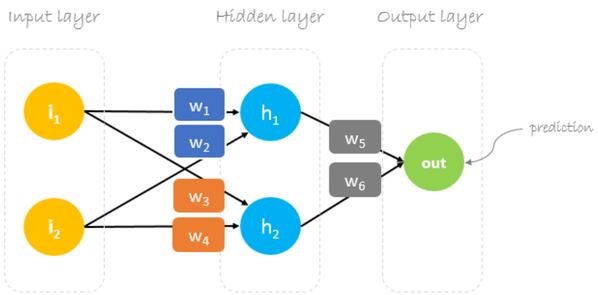
训练 MLP：反向传播



$$^*W_6 = W_6 - a\,\Delta h_2$$

https://hmkcode.com/ai/backpropagation-step-by-step/

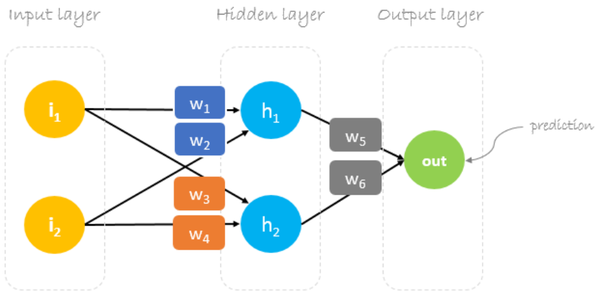
Training MLP: Backpropagation



$$^*W_6 = W_6 - a\,\Delta h_2$$

https://hmkcode.com/ai/backpropagation-step-by-step/

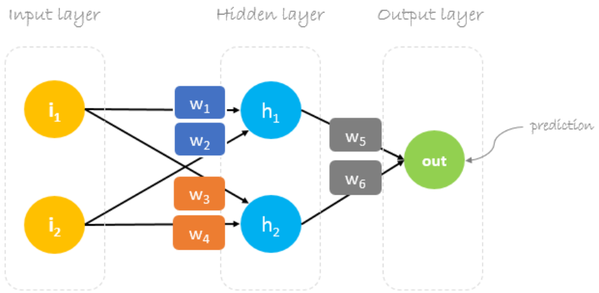
训练 MLP：反向传播



$$^*W_5 = W_5 - a\,\Delta h_1$$

https://hmkcode.com/ai/backpropagation-step-by-step/

Training MLP: Backpropagation



$$^*W_5 = W_5 - a\,\Delta h_1$$

https://hmkcode.com/ai/backpropagation-step-by-step/

训练 MLP：反向传播

$$\frac{\partial Error}{\partial W_1} = \frac{\partial Error}{\partial prediction} * \frac{\partial prediction}{\partial h_1} * \frac{\partial h_1}{\partial W_1}$$ ← chain rule

$$\frac{\partial Error}{\partial W_1} = \frac{\partial \frac{1}{2}(predictoin - actula)^2}{\partial prediciton} * \frac{\partial (h_1)\, w_5 + (h_2)\, w_6}{\partial h_1} * \frac{\partial\, i_1 w_1 + i_2 w_2}{\partial w_1}$$

$$\frac{\partial Error}{\partial W_1} = 2 * \frac{1}{2}(\mathbf{predictoin - actula}) \frac{\partial (predictoin - actula)}{\partial prediciton} * (w_5) * (i_1)$$

$$\frac{\partial Error}{\partial W_1} = (\mathbf{predictoin - actula}) * (w_5 i_1)$$ Δ = prediction – actual ← delta

$$\frac{\partial Error}{\partial W_1} = \Delta\, w_5 i_1$$

Error = $\frac{1}{2}$(prediction – actual)²

prediction = $(h_1)\, w_5 + (h_2)\, w_6$

$h_1 = i_1 w_1 + i_2 w_2$

https://hmkcode.com/ai/backpropagation-step-by-step/

Training MLP: Backpropagation

$$\frac{\partial Error}{\partial W_1} = \frac{\partial Error}{\partial prediction} * \frac{\partial prediction}{\partial h_1} * \frac{\partial h_1}{\partial W_1}$$ ← chain rule

$$\frac{\partial Error}{\partial W_1} = \frac{\partial \frac{1}{2}(predictoin - actula)^2}{\partial prediciton} * \frac{\partial (h_1)\, w_5 + (h_2)\, w_6}{\partial h_1} * \frac{\partial\, i_1 w_1 + i_2 w_2}{\partial w_1}$$

$$\frac{\partial Error}{\partial W_1} = 2 * \frac{1}{2}(\mathbf{predictoin - actula}) \frac{\partial (predictoin - actula)}{\partial prediciton} * (w_5) * (i_1)$$

$$\frac{\partial Error}{\partial W_1} = (\mathbf{predictoin - actula}) * (w_5 i_1)$$ Δ = prediction – actual ← delta

$$\frac{\partial Error}{\partial W_1} = \Delta\, w_5 i_1$$

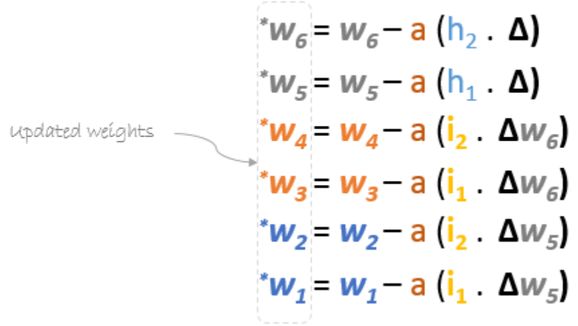
Error = $\frac{1}{2}$(prediction – actual)²

prediction = $(h_1)\, w_5 + (h_2)\, w_6$

$h_1 = i_1 w_1 + i_2 w_2$

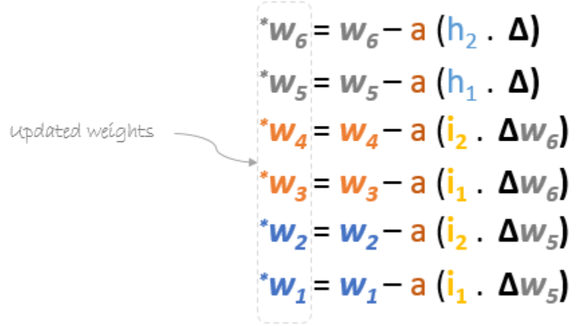
https://hmkcode.com/ai/backpropagation-step-by-step/

训练 MLP：反向传播



https://hmkcode.com/ai/backpropagation-step-by-step/

Training MLP: Backpropagation



https://hmkcode.com/ai/backpropagation-step-by-step/

训练 MLP：反向传播

$$\begin{bmatrix} w_5 \\ w_6 \end{bmatrix} = \begin{bmatrix} w_5 \\ w_6 \end{bmatrix} - a\,\Delta \begin{bmatrix} h_1 \\ h_2 \end{bmatrix} = \begin{bmatrix} w_5 \\ w_6 \end{bmatrix} - \begin{bmatrix} a h_1 \Delta \\ a h_2 \Delta \end{bmatrix}$$

$$\begin{bmatrix} w_1 & w_3 \\ w_2 & w_4 \end{bmatrix} = \begin{bmatrix} w_1 & w_3 \\ w_2 & w_4 \end{bmatrix} - a\,\Delta \begin{bmatrix} i_1 \\ i_2 \end{bmatrix} . [w_5 \quad w_6] = \begin{bmatrix} w_1 & w_3 \\ w_2 & w_4 \end{bmatrix} - \begin{bmatrix} a\,i_1 \Delta w_5 & a\,i_1 \Delta w_6 \\ a\,i_2 \Delta w_5 & a\,i_2 \Delta w_6 \end{bmatrix}$$



https://hmkcode.com/ai/backpropagation-step-by-step/

Training MLP: Backpropagation

$$\begin{bmatrix} w_5 \\ w_6 \end{bmatrix} = \begin{bmatrix} w_5 \\ w_6 \end{bmatrix} - a\,\Delta \begin{bmatrix} h_1 \\ h_2 \end{bmatrix} = \begin{bmatrix} w_5 \\ w_6 \end{bmatrix} - \begin{bmatrix} a h_1 \Delta \\ a h_2 \Delta \end{bmatrix}$$

$$\begin{bmatrix} w_1 & w_3 \\ w_2 & w_4 \end{bmatrix} = \begin{bmatrix} w_1 & w_3 \\ w_2 & w_4 \end{bmatrix} - a\,\Delta \begin{bmatrix} i_1 \\ i_2 \end{bmatrix} . [w_5 \quad w_6] = \begin{bmatrix} w_1 & w_3 \\ w_2 & w_4 \end{bmatrix} - \begin{bmatrix} a\,i_1 \Delta w_5 & a\,i_1 \Delta w_6 \\ a\,i_2 \Delta w_5 & a\,i_2 \Delta w_6 \end{bmatrix}$$



https://hmkcode.com/ai/backpropagation-step-by-step/

训练 MLP：反向传播

$\Delta = 0.191 - 1 = -0.809$ ← **Delta** = prediction - actual

$a = 0.05$ ← **Learning rate,** we smartly guess this number

$$\begin{bmatrix} w_5 \\ w_6 \end{bmatrix} = \begin{bmatrix} 0.14 \\ 0.15 \end{bmatrix} - 0.05(-0.809) \begin{bmatrix} 0.85 \\ 0.48 \end{bmatrix} = \begin{bmatrix} 0.14 \\ 0.15 \end{bmatrix} - \begin{bmatrix} -0.034 \\ -0.019 \end{bmatrix} = \begin{bmatrix} 0.17 \\ 0.17 \end{bmatrix}$$

$$\begin{bmatrix} w_1 & w_3 \\ w_2 & w_4 \end{bmatrix} = \begin{bmatrix} .11 & .12 \\ .21 & .08 \end{bmatrix} - 0.05(-0.809) \begin{bmatrix} 2 \\ 3 \end{bmatrix} . [0.14 \quad 0.15] = \begin{bmatrix} .11 & .12 \\ .21 & .08 \end{bmatrix} - \begin{bmatrix} -0.011 & -0.012 \\ -0.017 & -0.018 \end{bmatrix} = \begin{bmatrix} .12 & .13 \\ .23 & .10 \end{bmatrix}$$



https://hmkcode.com/ai/backpropagation-step-by-step/

Training MLP: Backpropagation

$\Delta = 0.191 - 1 = -0.809$ ← **Delta** = prediction - actual

$a = 0.05$ ← **Learning rate,** we smartly guess this number

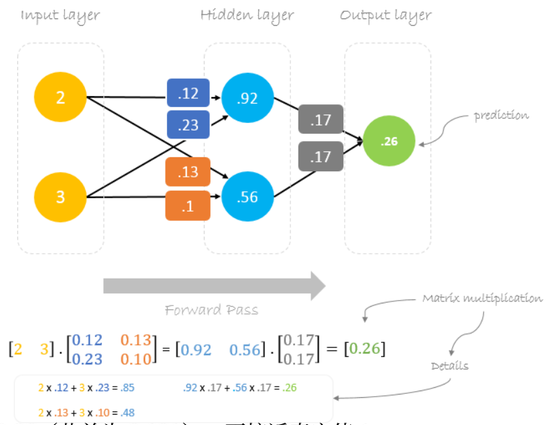
$$\begin{bmatrix} w_5 \\ w_6 \end{bmatrix} = \begin{bmatrix} 0.14 \\ 0.15 \end{bmatrix} - 0.05(-0.809) \begin{bmatrix} 0.85 \\ 0.48 \end{bmatrix} = \begin{bmatrix} 0.14 \\ 0.15 \end{bmatrix} - \begin{bmatrix} -0.034 \\ -0.019 \end{bmatrix} = \begin{bmatrix} 0.17 \\ 0.17 \end{bmatrix}$$

$$\begin{bmatrix} w_1 & w_3 \\ w_2 & w_4 \end{bmatrix} = \begin{bmatrix} .11 & .12 \\ .21 & .08 \end{bmatrix} - 0.05(-0.809) \begin{bmatrix} 2 \\ 3 \end{bmatrix} . [0.14 \quad 0.15] = \begin{bmatrix} .11 & .12 \\ .21 & .08 \end{bmatrix} - \begin{bmatrix} -0.011 & -0.012 \\ -0.017 & -0.018 \end{bmatrix} = \begin{bmatrix} .12 & .13 \\ .23 & .10 \end{bmatrix}$$



https://hmkcode.com/ai/backpropagation-step-by-step/

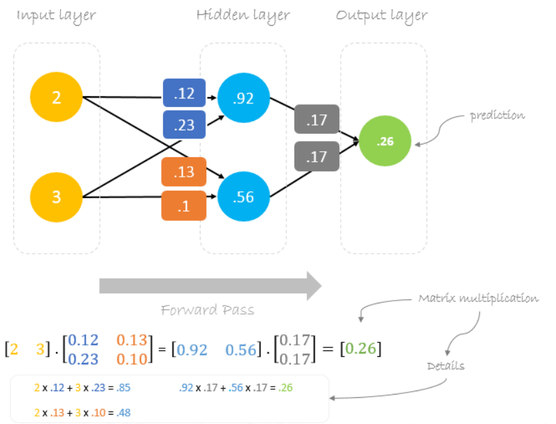
训练 MLP：反向传播



使用更新后的权重，预测值为 0.26（此前为 0.191），更接近真实值 1。

https://hmkcode.com/ai/backpropagation-step-by-step/

Training MLP: Backpropagation



With the update weights, the prediction is 0.26 (previous 0.191), closer to the ground truth 1.

https://hmkcode.com/ai/backpropagation-step-by-step/

演练场

- https://playground.tensorflow.org

- 使用默认设置查看其行为
- 将 Sigmoid 改为 tanh
- 使用 sigmoid/Tanh 和 5 层，以查看学习行为
- 将其改为 ReLU。

Playground

- https://playground.tensorflow.org

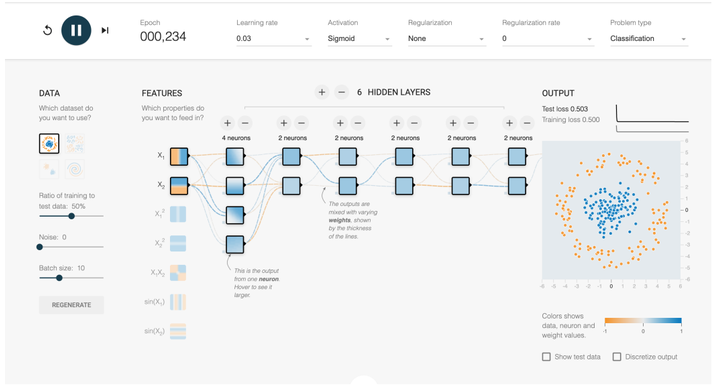
- Use default setting to see the behavior
- Change Sigmoid to tanh
- Use sigmoid/Tanh, and 5 layers, to see the learning behavior
- Change it to ReLU.

激活函数

Activation Function

一些观察

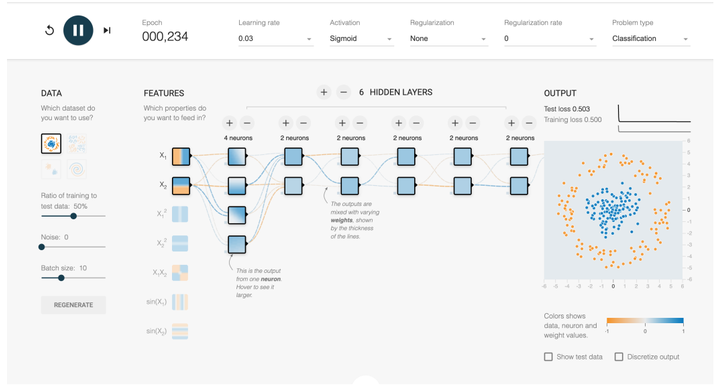
- 如果使用很多层，网络不会收敛



https://playground.tensorflow.org

Some observations

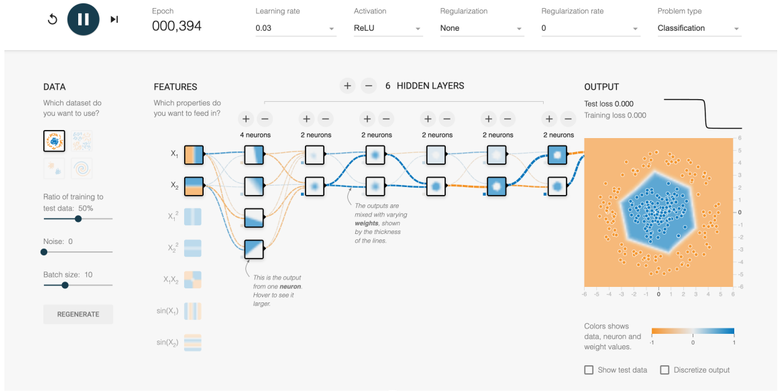
- The network don’t converge if we use a lot of layers



https://playground.tensorflow.org

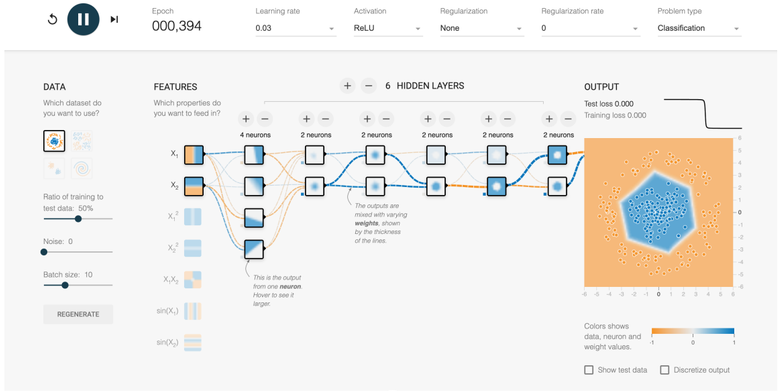
一些观察

- 将 Sigmoid 改为 ReLU 可解决此问题



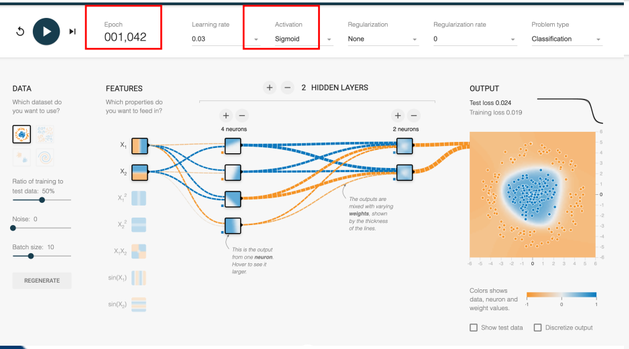
Some observations

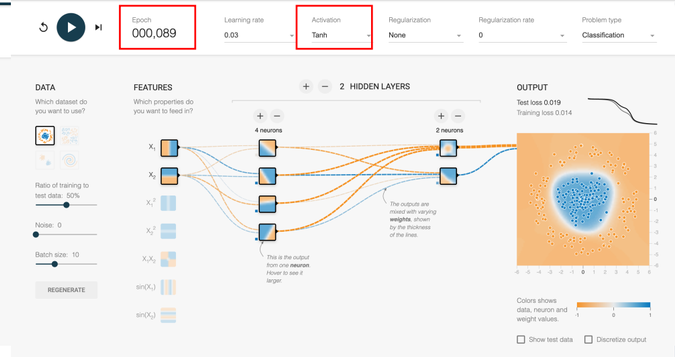
- Changing Sigmoid to ReLU solves this problem



一些观察

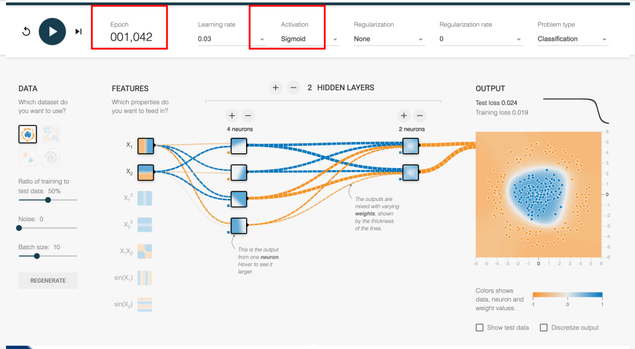
- Tanh 的学习速度比 sigmoid 更快

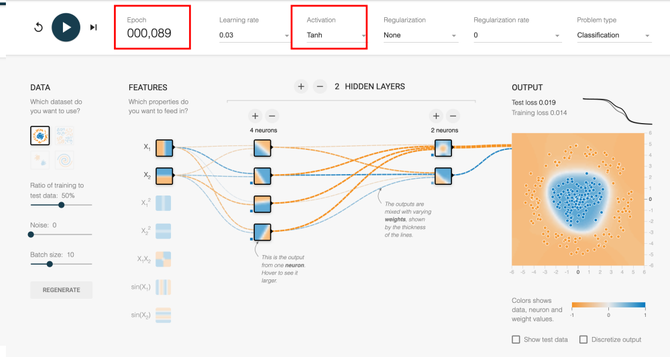




Some observations

- Tanh learns faster than sigmoid

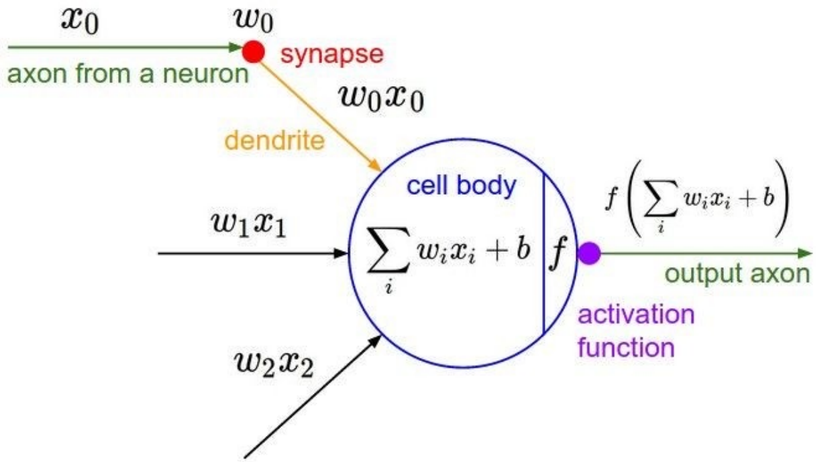




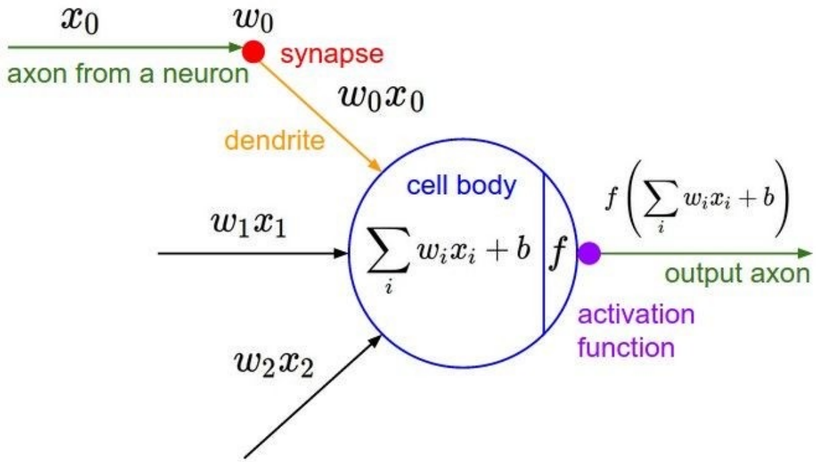
激活函数

Activation Functions

激活函数



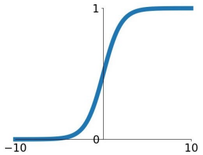
Activation Functions



激活函数

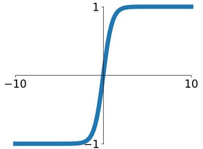
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



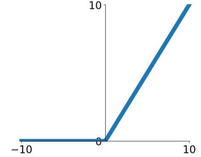
tanh

$$\tanh(x)$$



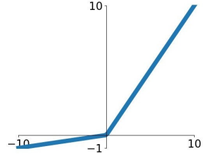
ReLU

$$\max(0, x)$$



Leaky ReLU

$$\max(0.1x, x)$$

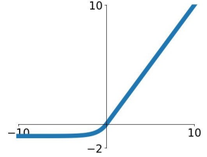


Maxout

$$\max(w_1^T x + b_1, w_2^T x + b_2)$$

ELU

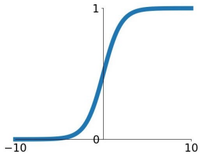
$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$



Activation Functions

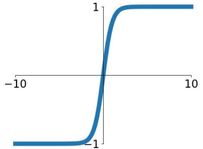
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



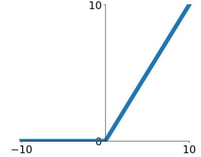
tanh

$$\tanh(x)$$



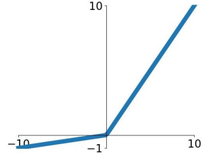
ReLU

$$\max(0, x)$$



Leaky ReLU

$$\max(0.1x, x)$$

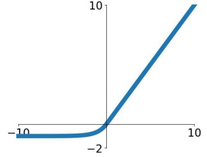


Maxout

$$\max(w_1^T x + b_1, w_2^T x + b_2)$$

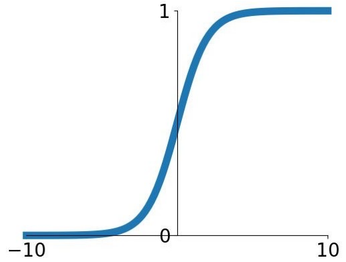
ELU

$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$



激活函数

$$\sigma(x) = 1/(1+e^{-x})$$

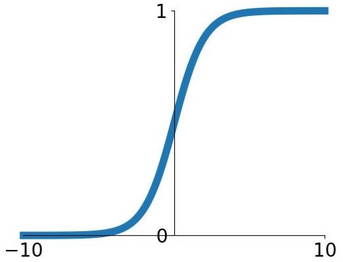


Sigmoid

- 将数值压缩到范围 0 - 历史上很受欢迎，因为它们可以很好地解释为神经元的饱和“发放率”

Activation Functions

$$\sigma(x) = 1/(1+e^{-x})$$

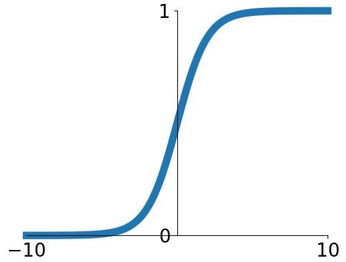


Sigmoid

- Squashes numbers to range [0,1]
- Historically popular since they have nice interpretation as a saturating “firing rate” of a neuron

激活函数

$$\sigma(x) = 1/(1+e^{-x})$$



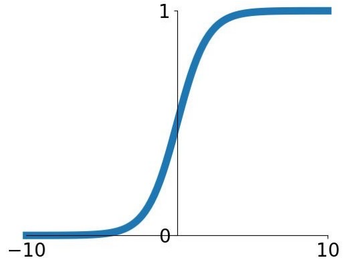
Sigmoid

- 将数值压缩到范围[0,1]，由于它们可以很好地解释为神经元的饱和“发放率”，因此历来很受欢迎

1. 饱和神经元会“杀死”梯度

Activation Functions

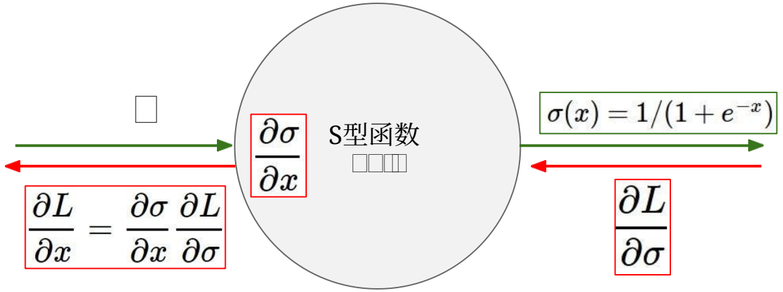
$$\sigma(x) = 1/(1+e^{-x})$$

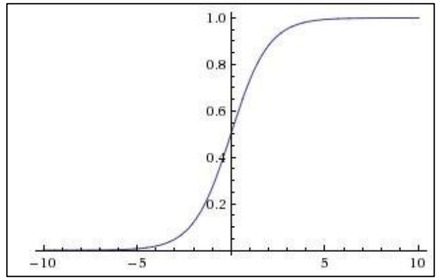


Sigmoid

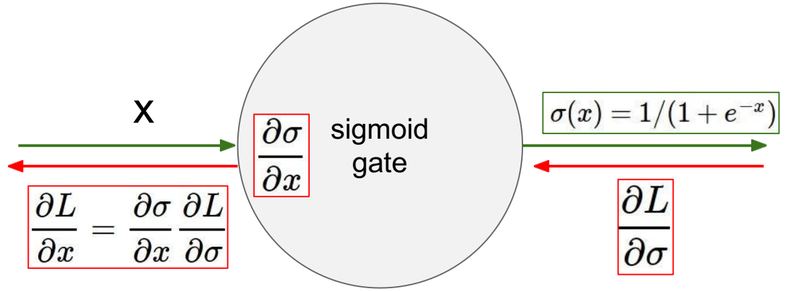
- Squashes numbers to range [0,1]
- Historically popular since they have nice interpretation as a saturating “firing rate” of a neuron

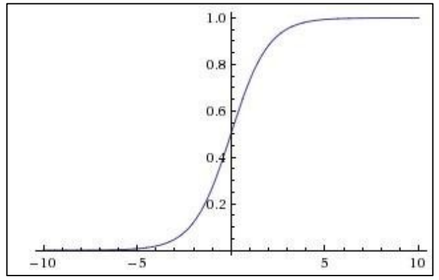
1. Saturated neurons “kill” the gradients



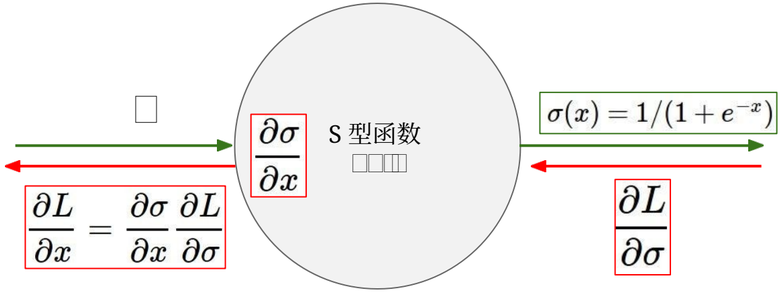


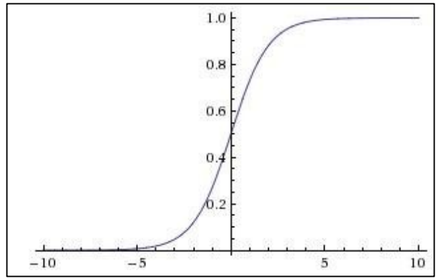
$$\frac{\partial \sigma(x)}{\partial x} = \sigma(x)\,(1-\sigma(x))$$





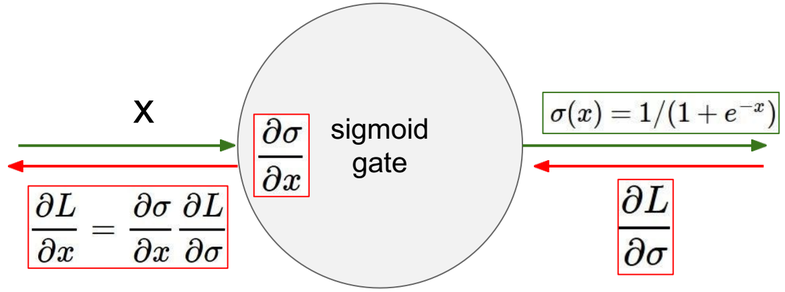
$$\frac{\partial \sigma(x)}{\partial x} = \sigma(x)\,(1-\sigma(x))$$

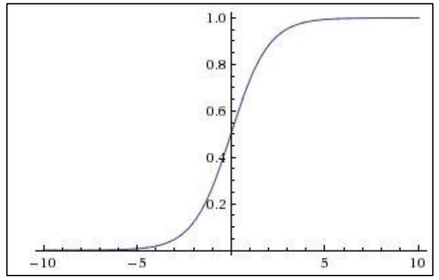




当 x ? -10 时会发生什么?

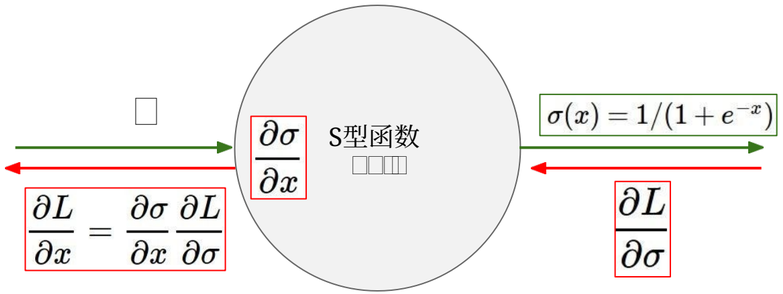
$$\frac{\partial \sigma(x)}{\partial x} = \sigma(x)\,(1-\sigma(x))$$

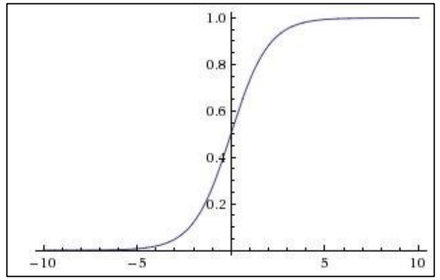




What happens when x = -10?

$$\frac{\partial \sigma(x)}{\partial x} = \sigma(x)\,(1-\sigma(x))$$



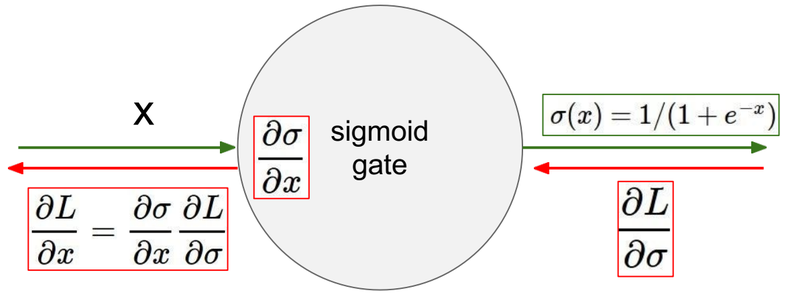


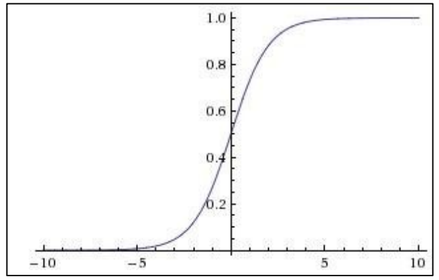
当 x ? -10 时会发生什么?

$$\sigma(x) = \sim 0$$

$$\frac{\partial \sigma(x)}{\partial x} = \sigma(x)\,(1-\sigma(x)\,) = 0(1-0) = 0$$

$$\frac{\partial \sigma(x)}{\partial x} = \sigma(x)\,(1-\sigma(x)\,)$$



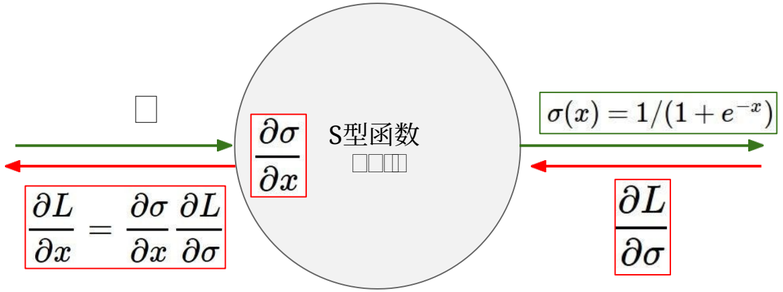


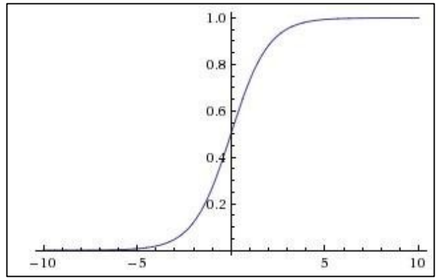
What happens when x = -10?

$$\sigma(x) = \sim 0$$

$$\frac{\partial \sigma(x)}{\partial x} = \sigma(x)\,(1-\sigma(x)\,) = 0(1-0) = 0$$

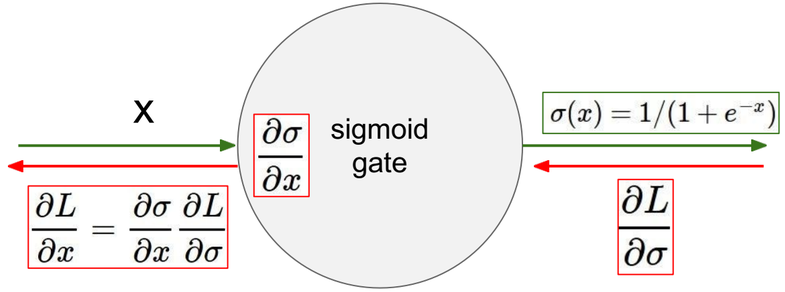
$$\frac{\partial \sigma(x)}{\partial x} = \sigma(x)\,(1-\sigma(x)\,)$$

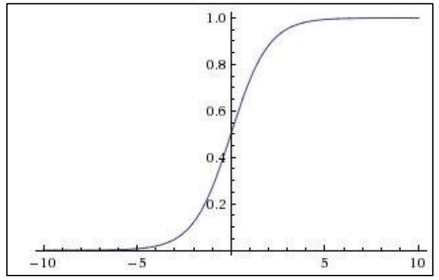




当 x ?-10? 当 x ? 0?

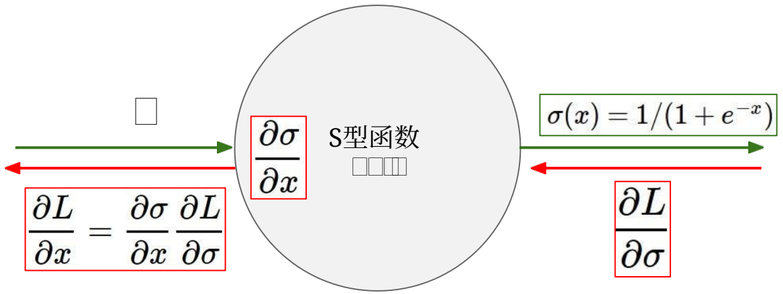
$$\frac{\partial \sigma(x)}{\partial x} = \sigma(x)\,(1-\sigma(x))$$

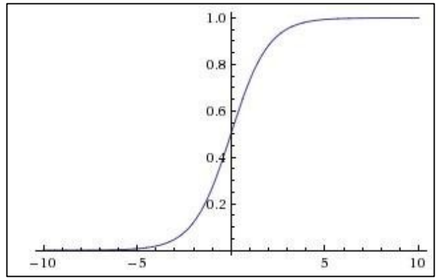




What happens when x = -10?
What happens when x = 0?

$$\frac{\partial \sigma(x)}{\partial x} = \sigma(x)\,(1-\sigma(x))$$

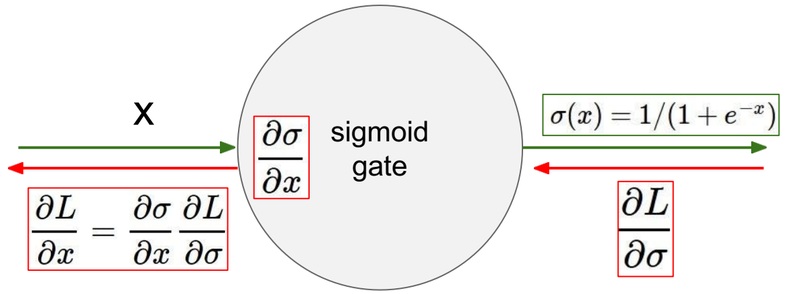


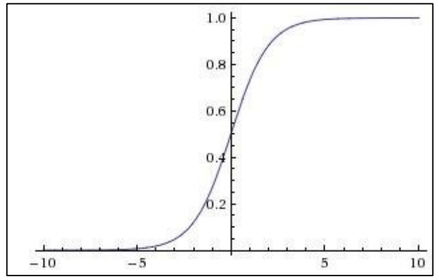


当 x ?-10? 当 x ? 0? 当 x ? 10?

$$\frac{\partial \sigma(x)}{\partial x} = \sigma(x)\,(1-\sigma(x))$$

$\sigma(x) = \sim 1$ $\frac{\partial \sigma(x)}{\partial x} = \sigma(x)\,(1-\sigma(x)) = 1(1-1) = 0$

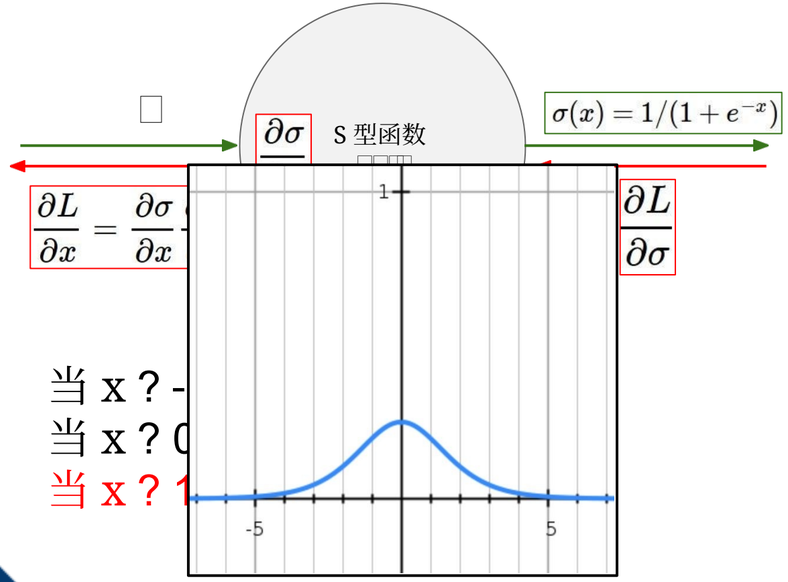


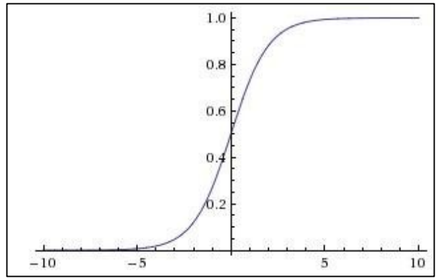


What happens when x = -10?
What happens when x = 0?
What happens when x = 10?

$$\frac{\partial \sigma(x)}{\partial x} = \sigma(x)\,(1-\sigma(x))$$

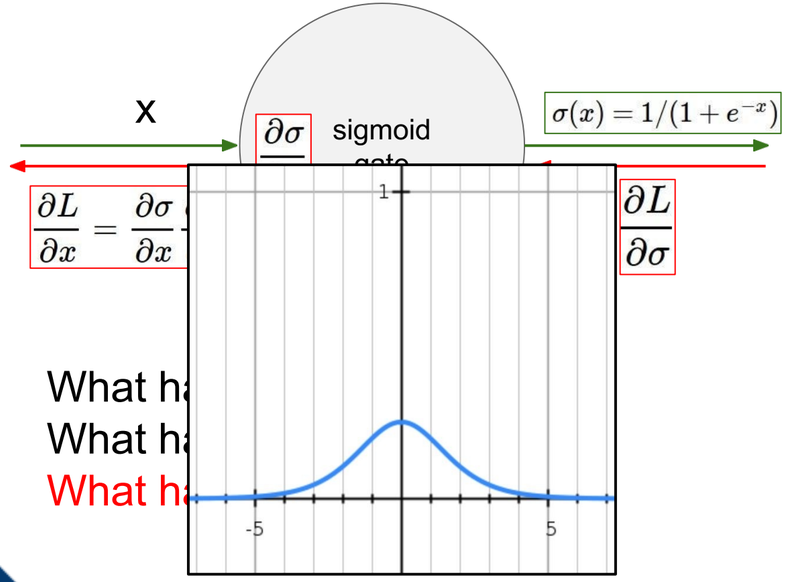
$\sigma(x) = \sim 1$ $\frac{\partial \sigma(x)}{\partial x} = \sigma(x)\,(1-\sigma(x)) = 1(1-1) = 0$

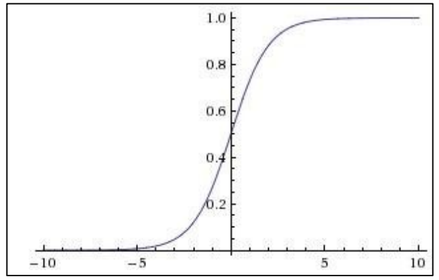




$\frac{\partial \sigma(x)}{\partial x} = \sigma(x)\,(1-\sigma(x))$

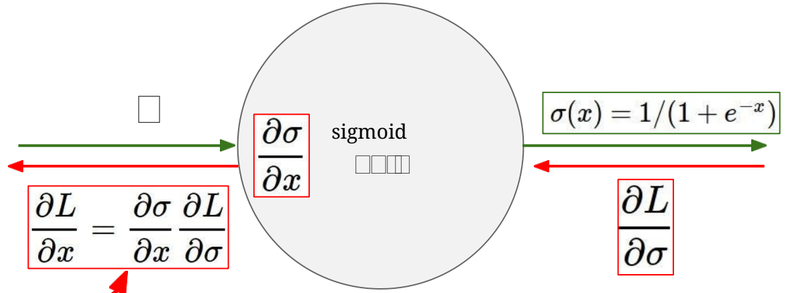
梯度

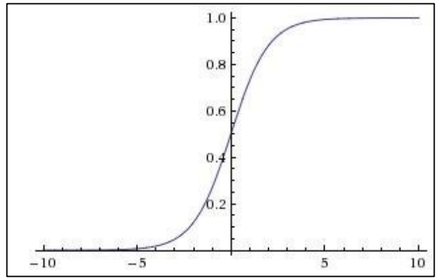




$\frac{\partial \sigma(x)}{\partial x} = \sigma(x)\,(1-\sigma(x))$

Gradient

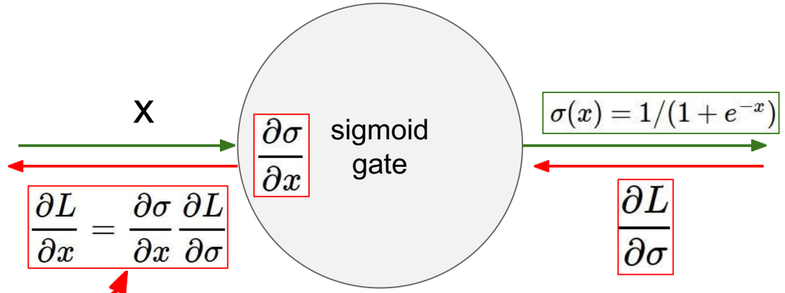


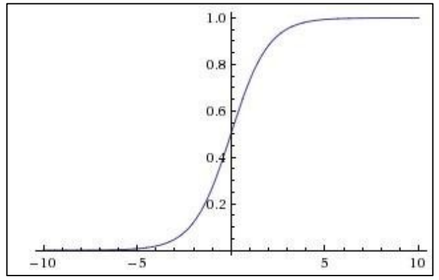


$$\frac{\partial \sigma(x)}{\partial x} = \sigma(x)\,(1-\sigma(x))$$

为什么这是一个问题?

如果反向流动的所有梯度都为零，权重将永远不会改变



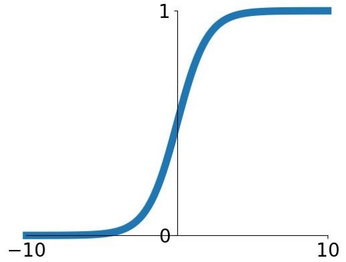


$$\frac{\partial \sigma(x)}{\partial x} = \sigma(x)\,(1-\sigma(x))$$

Why is this a problem?

If all the gradients flowing back will be zero and weights will never change

激活函数



Sigmoid

$$\sigma(x) = 1/(1+e^{-x})$$

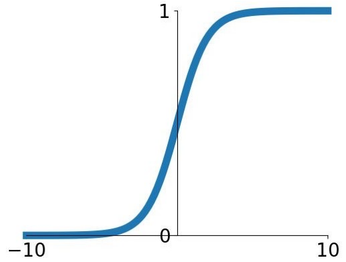
- 将数值压缩到范围□0□□

- 历史上很受欢迎，因为它们可以被很好地解释为神经元的饱和“发放率”

1. 饱和神经元会“杀死”梯度

2. Sigmoid 输出不是零中心的为什么这不好?

Activation Functions



Sigmoid

$$\sigma(x) = 1/(1+e^{-x})$$

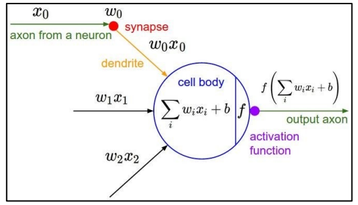
- Squashes numbers to range [0,1]
- Historically popular since they have nice interpretation as a saturating “firing rate” of a neuron

1. Saturated neurons “kill” the gradients
2. Sigmoid outputs are not zero-centered

Why this is bad?

考虑当神经元的输入始终为正时会发生什么.....

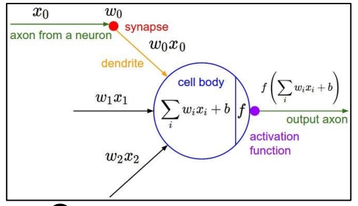
$$f\left(\sum_i w_i x_i + b\right)$$



关于 **w** 上的梯度，我们能说什么?

Consider what happens when the input to a neuron is always positive...

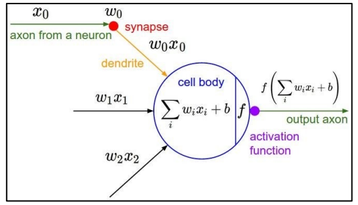
$$f\left(\sum_i w_i x_i + b\right)$$



What can we say about the gradients on **w**?

考虑当一个神经元的输入始终为正时会发生什么.....

$$f\left(\sum_i w_i x_i + b\right)$$

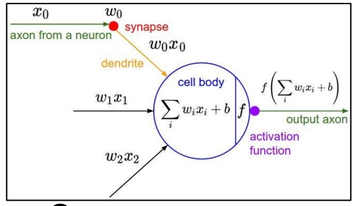


关于 **w** 上的梯度，我们能说什么?

$$\frac{\partial L}{\partial w} = \sigma(\sum_i w_i x_i + b)(1 - \sigma(\sum_i w_i x_i + b))x \times upstream_gradient$$

Consider what happens when the input to a neuron is always positive...

$$f\left(\sum_i w_i x_i + b\right)$$

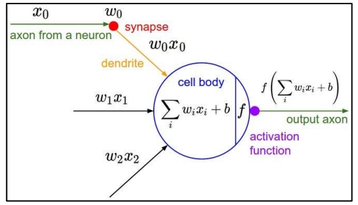


What can we say about the gradients on **w**?

$$\frac{\partial L}{\partial w} = \sigma(\sum_i w_i x_i + b)(1 - \sigma(\sum_i w_i x_i + b))x \times upstream_gradient$$

考虑当一个神经元的输入始终为正时会发生什么.....

$$f\left(\sum_i w_i x_i + b\right)$$



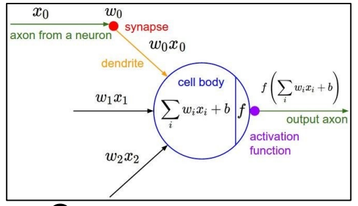
关于 **w** 的梯度，我们能说些什么?

我们知道 sigmoid 的局部梯度始终为正

$$\frac{\partial L}{\partial w} = \sigma\left(\sum_i w_i x_i + b\right)\left(1 - \sigma\left(\sum_i w_i x_i + b\right)\right) x \times upstream_gradient$$

Consider what happens when the input to a neuron is always positive...

$$f\left(\sum_i w_i x_i + b\right)$$



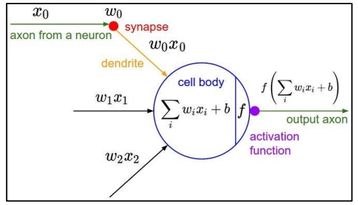
What can we say about the gradients on **w**?

We know that local gradient of sigmoid is always positive

$$\frac{\partial L}{\partial w} = \sigma\left(\sum_i w_i x_i + b\right)\left(1 - \sigma\left(\sum_i w_i x_i + b\right)\right) x \times upstream_gradient$$

考虑当一个神经元的输入始终为正时会发生什么.....

$$f\left(\sum_i w_i x_i + b\right)$$



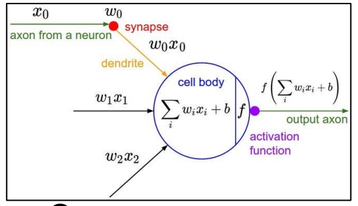
关于 **w** 的梯度，我们能说些什么?

我们知道 sigmoid 函数的局部梯度始终为正 我们假设 x 始终为正

$$\frac{\partial L}{\partial w} = \sigma\left(\sum_i w_i x_i + b\right)\left(1 - \sigma\left(\sum_i w_i x_i + b\right)\right) x \times upstream_gradient$$

Consider what happens when the input to a neuron is always positive...

$$f\left(\sum_i w_i x_i + b\right)$$



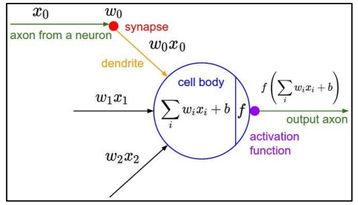
What can we say about the gradients on **w**?

We know that local gradient of sigmoid is always positive
We are assuming x is always positive

$$\frac{\partial L}{\partial w} = \sigma\left(\sum_i w_i x_i + b\right)\left(1 - \sigma\left(\sum_i w_i x_i + b\right)\right) x \times upstream_gradient$$

考虑当神经元的输入始终为正时会发生什么.....

$$f\left(\sum_i w_i x_i + b\right)$$



关于 **w** 的梯度，我们能说些什么?

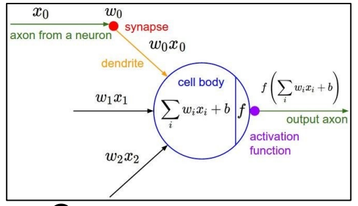
我们知道 sigmoid 的局部梯度始终为正 我们假设 x 始终为正

所以！！梯度的符号 **对于所有 w_i** 与上游标量梯度的符号相同 ！

$$\frac{\partial L}{\partial w} = \sigma\left(\sum_i w_i x_i + b\right)\left(1 - \sigma\left(\sum_i w_i x_i + b\right)\right)x \times upstream_gradient$$

Consider what happens when the input to a neuron is always positive...

$$f\left(\sum_i w_i x_i + b\right)$$



What can we say about the gradients on **w**?

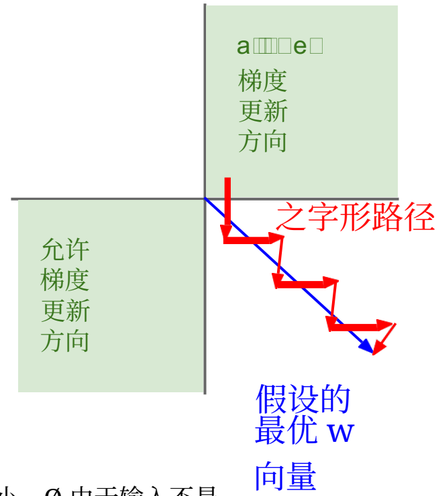
We know that local gradient of sigmoid is always positive
We are assuming x is always positive

So!! Sign of gradient **for all w_i** is the same as the sign of upstream scalar gradient!

$$\frac{\partial L}{\partial w} = \sigma\left(\sum_i w_i x_i + b\right)\left(1 - \sigma\left(\sum_i w_i x_i + b\right)\right)x \times upstream_gradient$$

考虑当神经元的输入始终为正时会发生什么.....

$$f\left(\sum_i w_i x_i + b\right)$$



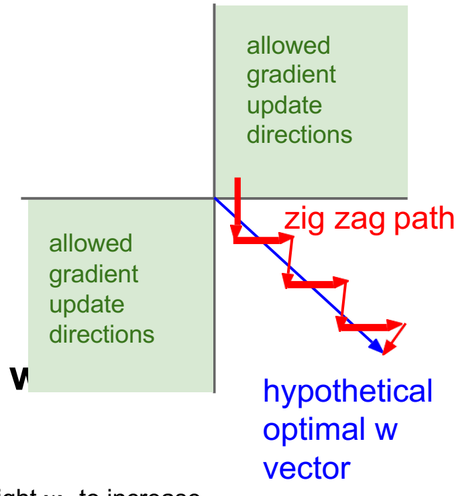
关于 **w** 上的梯度，我们能说些什么?

始终全为正或全为负

Ø 想象这样一个场景：最优解要求权重 w_1 增大，但权重 w_2 减小。Ø 由于输入不是零中心的，网络 **无法同时做出这些相反的调整**。它被迫在单步中要么增大所有权重，要么减小所有权重。

Consider what happens when the input to a neuron is always positive...

$$f\left(\sum_i w_i x_i + b\right)$$



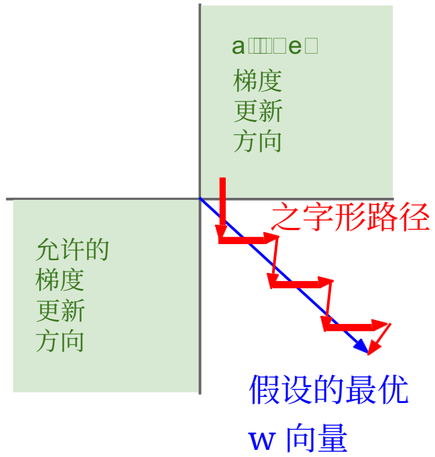
What can we say about the gradients on **w**

Always all positive or all negative

- Imagine a scenario where the optimal solution requires weight w_1 to increase, but weight w_2 to decrease.
- Because the inputs are not zero-centered, the network **cannot make these opposite adjustments at the same time**. It is forced to either increase all weights or decrease all weights in a single step.

考虑当神经元的输入始终为正时会发生什么...

$$f\left(\sum_i w_i x_i + b\right)$$



关于 **w** 的梯度，我们能说些什么?

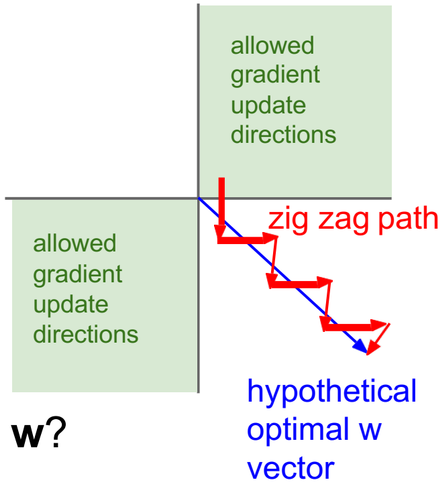
总是全为正或全为负 :(

如何改进?

对于单个元素！小批量会有帮助

Consider what happens when the input to a neuron is always positive...

$$f\left(\sum_i w_i x_i + b\right)$$



What can we say about the gradients on **w**?

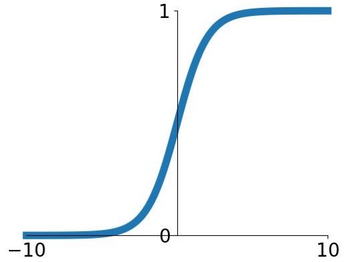
Always all positive or all negative :(

How to improve?

For a single element! Minibatches help

激活函数

$$\sigma(x) = 1/(1+e^{-x})$$



Sigmoid

- 将数值压缩到范围[0,1]

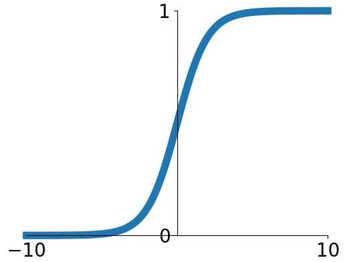
- 历史上很受欢迎，因为它们可以很好地解释为神经元的饱和“发放率”

1. 饱和神经元会“杀死”梯度

2. 输出不是零中心的

Activation Functions

$$\sigma(x) = 1/(1+e^{-x})$$

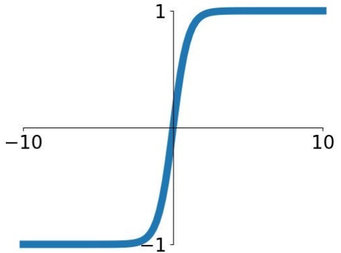


Sigmoid

- Squashes numbers to range [0,1]
- Historically popular since they have nice interpretation as a saturating “firing rate” of a neuron

1. Saturated neurons “kill” the gradients
2. outputs are not zero-centered

激活函数

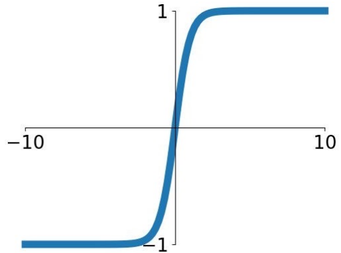


tanh(x)

- 将数值压缩到范围 z1 1□ 零中心化（很好） - 饱和时仍会扼杀梯度:(

[LeCun et al., 6NN6□

Activation Functions

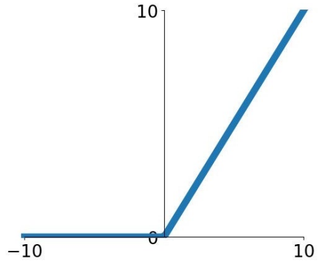


tanh(x)

- Squashes numbers to range [-1,1]
- zero centered (nice)
- still kills gradients when saturated :(

[LeCun et al., 1991]

激活函数



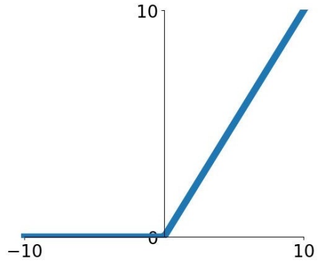
ReLU
（修正线性单元）

- 计算 **f(x) hmax(0,x)**

- 不会饱和（在 n区域） - 计算效率非常高 - 在实践中比 sigmoid/tanh 收敛快得多（例如6倍）

[Krizhevsky et al., 7567□

Activation Functions



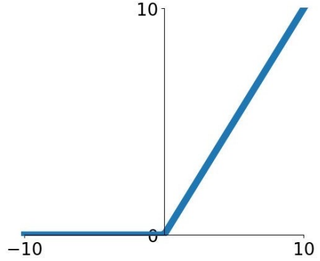
ReLU
(Rectified Linear Unit)

- Computes **f(x) = max(0,x)**

- Does not saturate (in +region)
- Very computationally efficient
- Converges much faster than sigmoid/tanh in practice (e.g. 6x)

[Krizhevsky et al., 2012]

激活函数



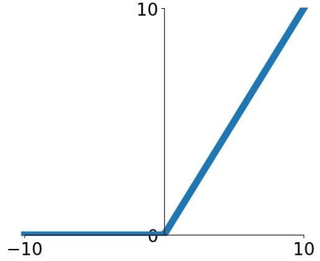
ReLU
（修正线性单元）

- 计算 **f(x) hmax(0,x)**

- 不会饱和（在 n区域） - 计算效率非常高 - 在实践中比 sigmoid/tanh 收敛快得多（例如 6 倍）

- 输出不以零为中心

Activation Functions



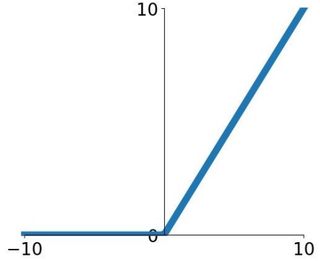
ReLU
(Rectified Linear Unit)

- Computes **f(x) = max(0,x)**

- Does not saturate (in +region)
- Very computationally efficient
- Converges much faster than sigmoid/tanh in practice (e.g. 6x)

- Not zero-centered output

激活函数



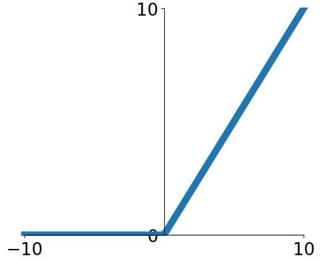
ReLU
（修正线性单元）

- 计算 **f(x) hmax(0,x)**

- 不会饱和（在 n区域） - 计算效率非常高 - 在实践中比 sigmoid/tanh 收敛快得多（例如 6 倍）

- 输出非零中心化 - 一个恼人的问题：死亡神经元

Activation Functions

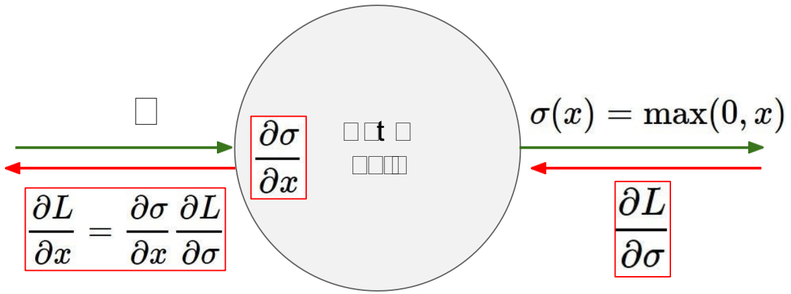


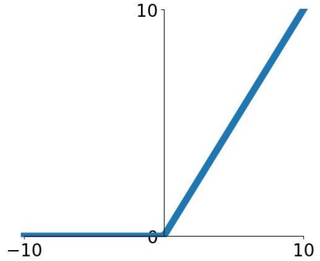
ReLU
(Rectified Linear Unit)

- Computes **f(x) = max(0,x)**

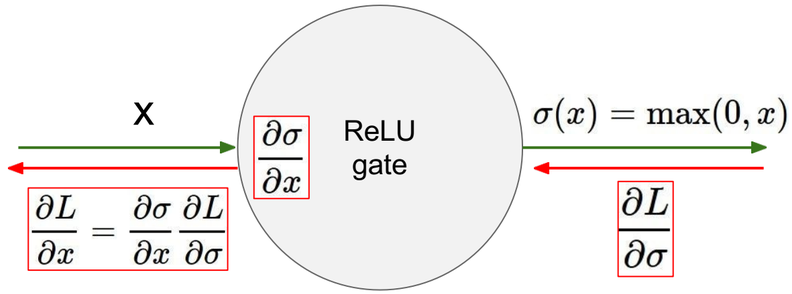
- Does not saturate (in +region)
- Very computationally efficient
- Converges much faster than sigmoid/tanh in practice (e.g. 6x)

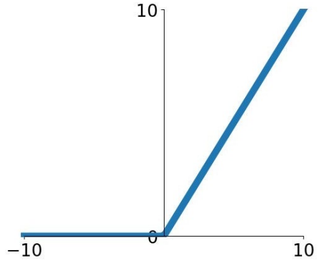
- Not zero-centered output
- An annoyance: dead neuron



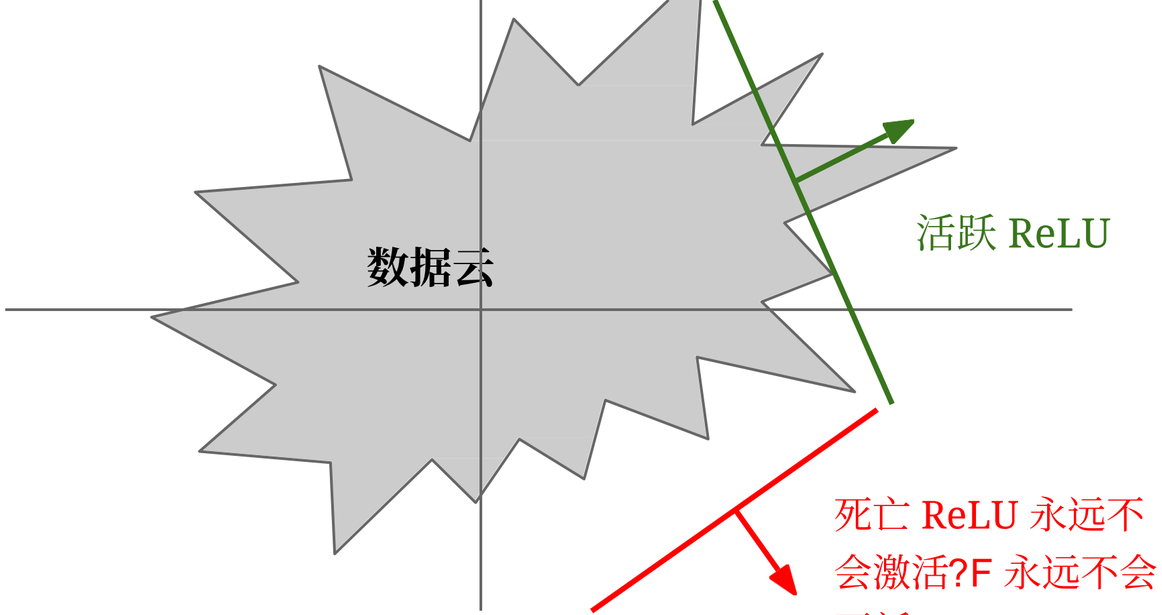


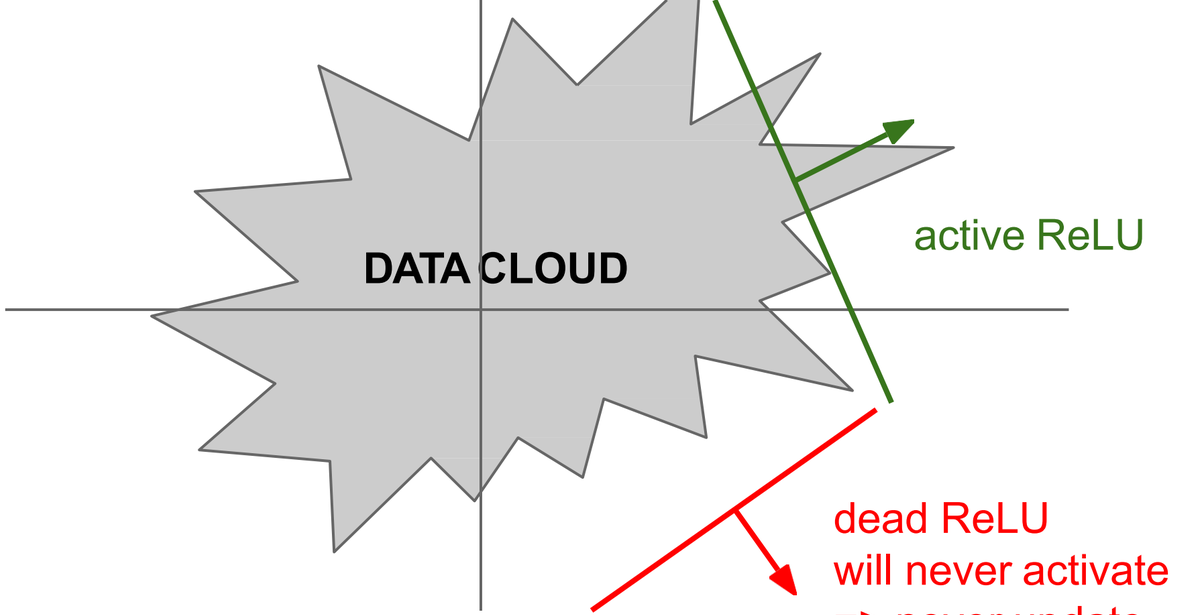
当 x ?-10 时会发生什么？ 当 x ? 0 时会发生什么？ 当 x ? 10 时会发生什么?

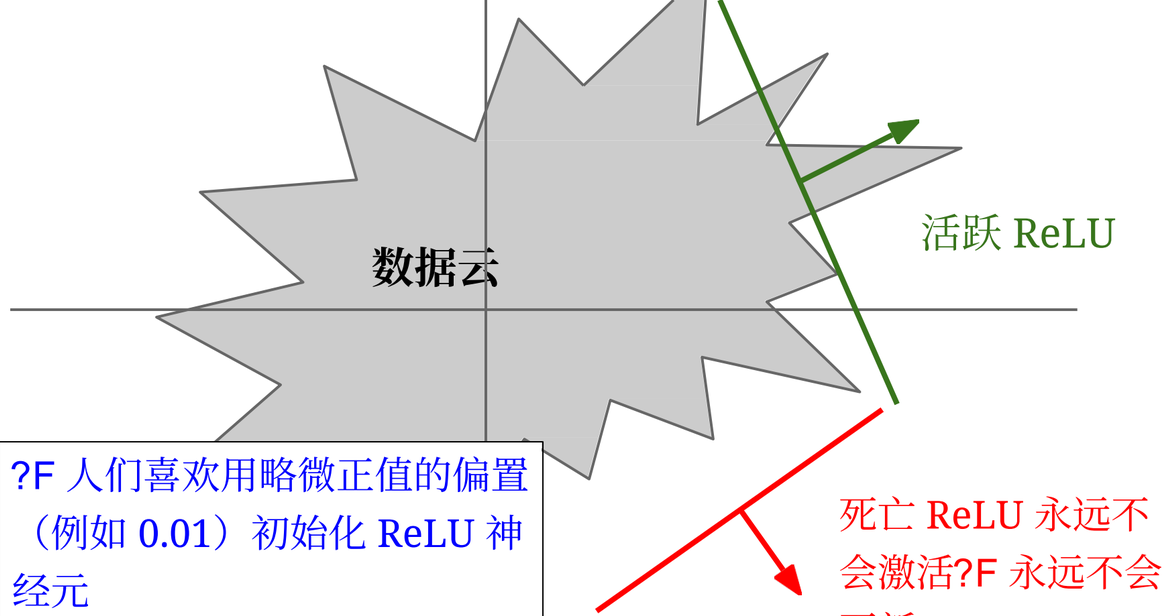




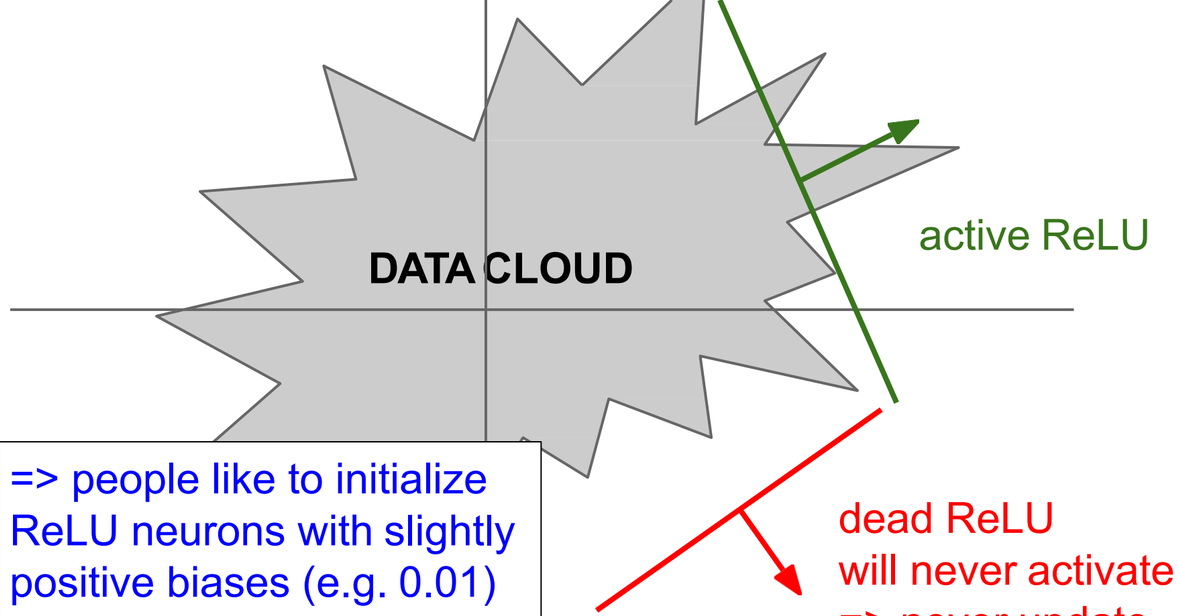
What happens when x = -10?
What happens when x = 0?
What happens when x = 10?


数据云
活跃 ReLU
死亡 ReLU 永远不会激活?F 永远不会更新


DATA CLOUD
active ReLU
dead ReLU
will never activate
=> never update

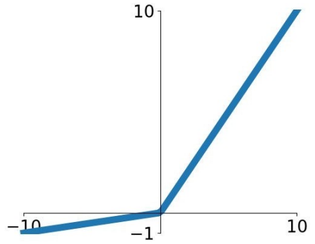


?F 人们喜欢用略微正值的偏置（例如 0.01）初始化 ReLU 神经元



=> people like to initialize ReLU neurons with slightly positive biases (e.g. 0.01)

激活函数

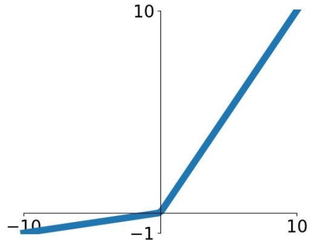


- 不会饱和 - 计算效率高 - 在实践中比 sigmoid/tanh 收敛快得多！（例如 6x） - **不会“死亡”**。

泄漏型 ReLU

$$f(x) = \max(0.01x, x)$$

Activation Functions

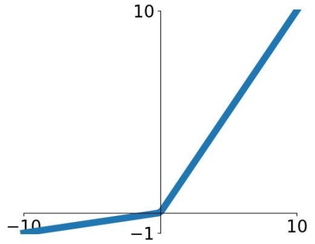


- Does not saturate
- Computationally efficient
- Converges much faster than sigmoid/tanh in practice! (e.g. 6x)
- **will not “die”.**

Leaky ReLU

$$f(x) = \max(0.01x, x)$$

激活函数



- 不会饱和 - 计算高效 - 在实践中比 sigmoid/tanh 收敛快得多！（例如 6 倍） - **不会“死亡”**。

泄漏型 ReLU

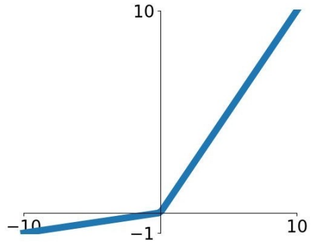
$$f(x) = \max(0.01x, x)$$

参数化整流器（PReLU）

$$f(x) = \max(\alpha x, x)$$

反向传播到 \alpha（参数）

Activation Functions



- Does not saturate
- Computationally efficient
- Converges much faster than sigmoid/tanh in practice! (e.g. 6x)
- **will not “die”.**

Leaky ReLU

$$f(x) = \max(0.01x, x)$$

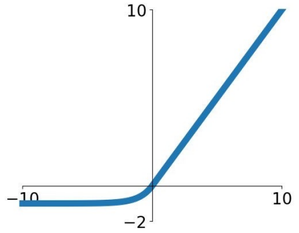
Parametric Rectifier (PReLU)

$$f(x) = \max(\alpha x, x)$$

backprop into \alpha (parameter)

激活函数

指数线性单元（ELU）



- 具备 ReLU 的所有优点 - 输出均值更接近零 - 与 Leaky ReLU 相比，负饱和区域增强了对噪声的鲁棒性

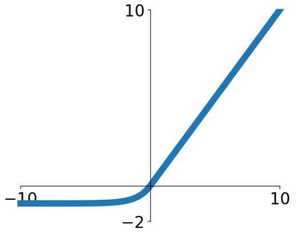
$$f(x) = \begin{cases} x & \text{if } x > 0 \\ \alpha\left(\exp(x) - 1\right) & \text{if } x \leq 0 \end{cases}$$

- 计算需要 exp()

(Alpha 默认值 UR)

Activation Functions

Exponential Linear Units (ELU)



- All benefits of ReLU
- Closer to zero mean outputs
- Negative saturation regime compared with Leaky ReLU adds some robustness to noise

$$f(x) = \begin{cases} x & \text{if } x > 0 \\ \alpha\left(\exp(x) - 1\right) & \text{if } x \leq 0 \end{cases}$$

- Computation requires exp()

(Alpha default = 1)

Maxout“神经元”

- 不具备点积的基本形式 ->逐元素非线性

- 泛化了 ReLU 与 Leaky ReLU
- 线性区域。不会饱和。不会死亡。

$$\max(w_1^T x + b_1, w_2^T x + b_2)$$

问题：参数/神经元数量翻倍 :(

Maxout “Neuron”

- Does not have the basic form of dot product -> element-wise nonlinearity
- Generalizes ReLU and Leaky ReLU
- Linear Regime. Does not saturate. Does not die.

$$\max(w_1^T x + b_1, w_2^T x + b_2)$$

Problem: doubles the number of parameters/neuron :(

在实践中：

- 使用 Remu。注意你的学习率 es
- 尝试 mea□y Remu 2n a□out 2h mu 以挤出一些边际收益 - 不要使用 si□moid 或 tanh 在隐藏层中- 它们可用于输出层以控制 trol 输出范围

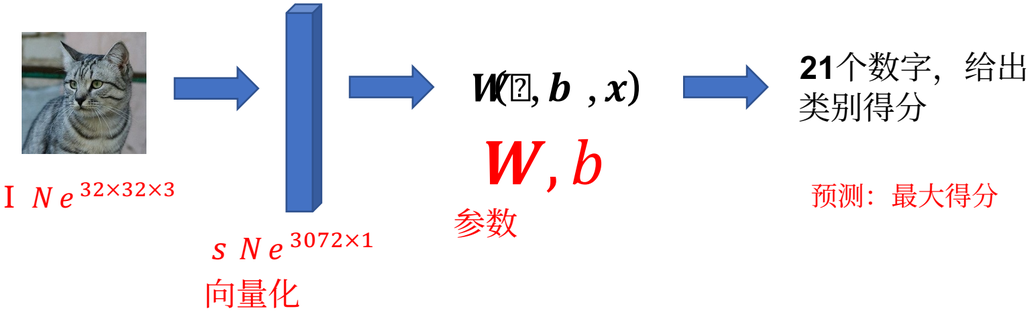
In practice:

- Use ReLU. Be careful with your learning rates
- Try out Leaky ReLU / Maxout / ELU to squeeze out some marginal gains
- Don’t use sigmoid or tanh in hidden layers
  - they can be used in output layers to control the output range

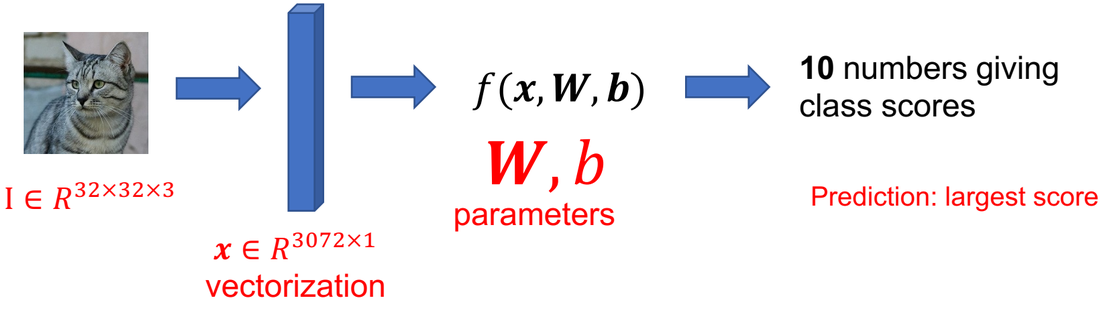
卷积

Convolution

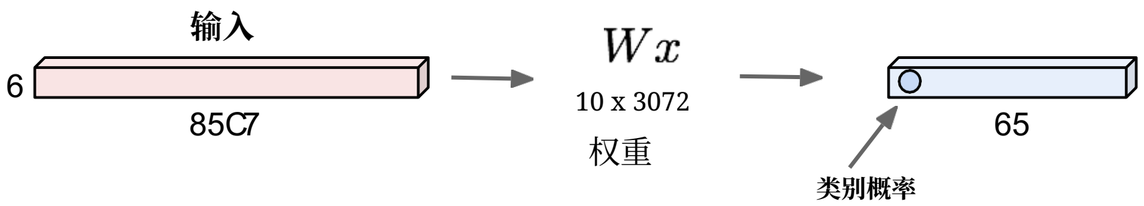
上一页：图像分类的线性模型



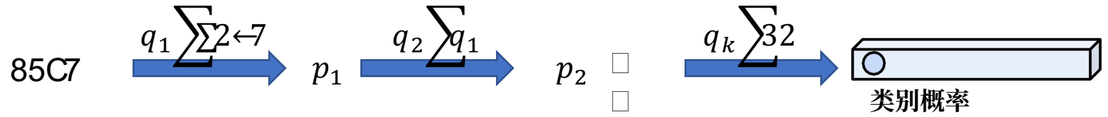
Previous: Linear model for image classification



上一节：图像分类的线性模型

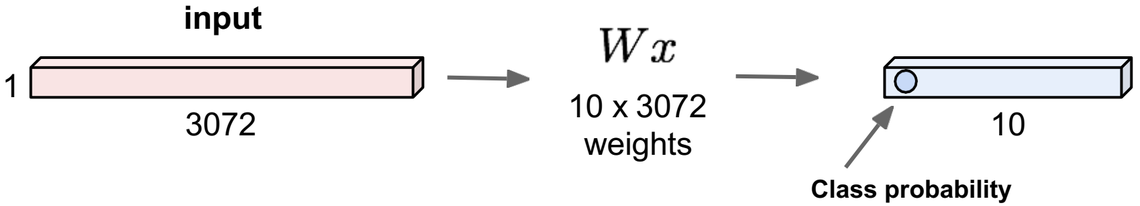


堆叠多个层以形成多层感知机：

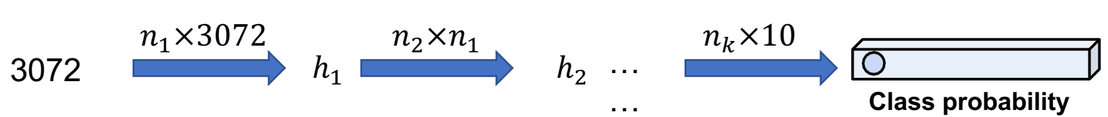


模型需要学习大量参数

Previous: Linear model for image classification

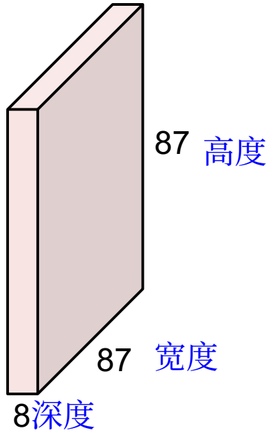


Stacking many layers to form the multi-layer perception:

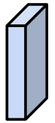


The model needs to learn a large number of parameters

卷积层



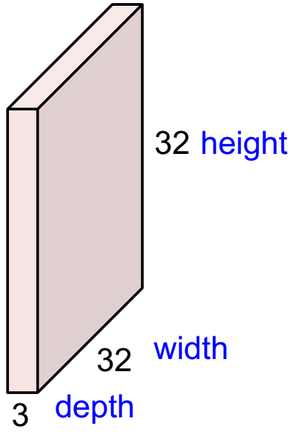
5x5x3 滤波器



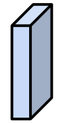
将滤波器与图像进行**卷积**，即“在空间上滑过图像，计算点积”

参数数量：$5\times5\times3=45$

Convolutional layers



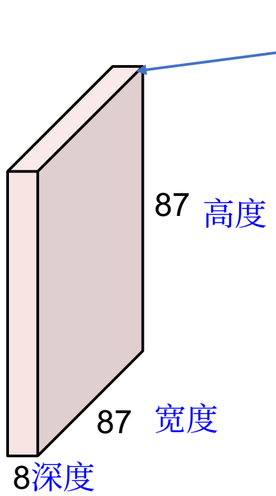
5x5x3 filter



Convolve the filter with the image
i.e. “slide over the image spatially, computing dot products"

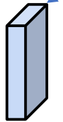
Number of parameters:
$5\times5\times3=45$

卷积层



滤波器始终贯穿输入体积的整个深度

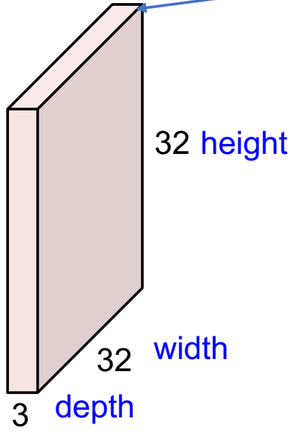
5x5x3 滤波器



将滤波器与图像进行**卷积**，即“在图像上空间滑动，计算点积

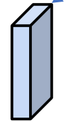
参数数量：$5\times5\times3=45$

Convolutional layers



Filters always extend the full depth of the input volume

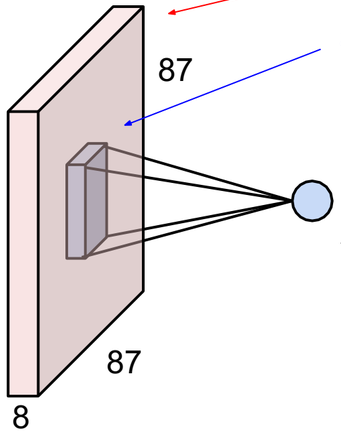
5x5x3 filter



Convolve the filter with the image
i.e. “slide over the image spatially, computing dot products

Number of parameters:
$5\times5\times3=45$

卷积层



32x32x3 图像

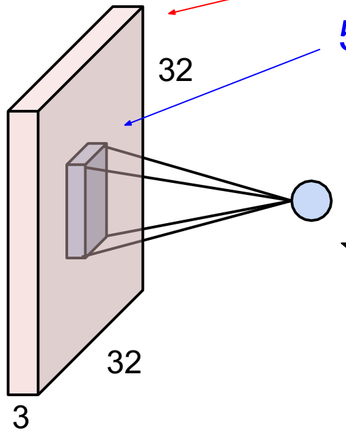
5x5x3 滤波器w

1 个数值：

将滤波器与图像中一个小的 5x5x3 块进行点积所得到的结果（即 5*5*8 RC= 维点积 +偏置）

$$w^T x + b$$

Convolution Layer



32x32x3 image

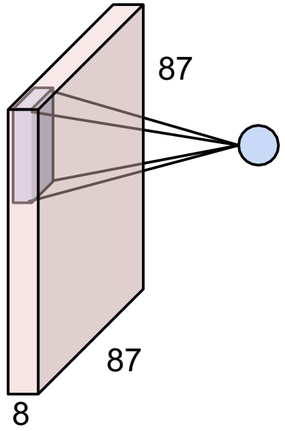
5x5x3 filter w

1 number:

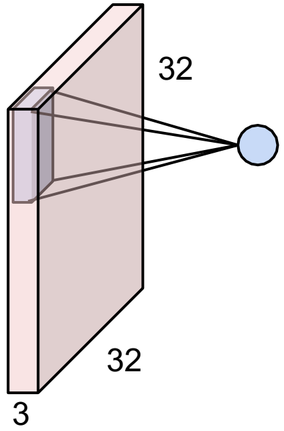
the result of taking a dot product between the filter and a small 5x5x3 chunk of the image (i.e. 5*5*3 = 75-dimensional dot product + bias)

$$w^T x + b$$

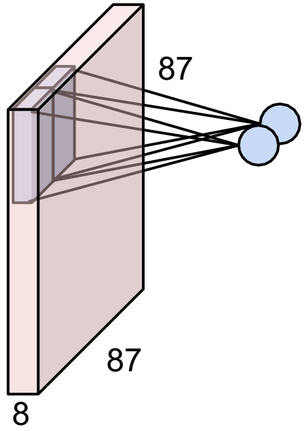
卷积层



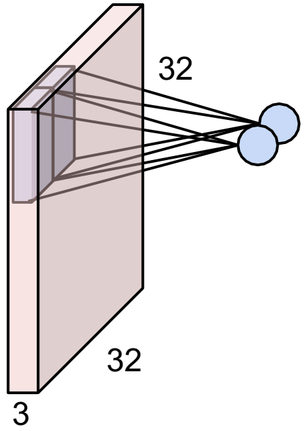
Convolution Layer



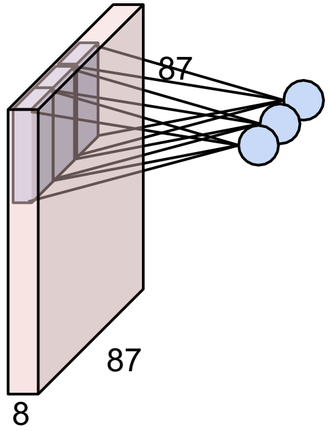
卷积层



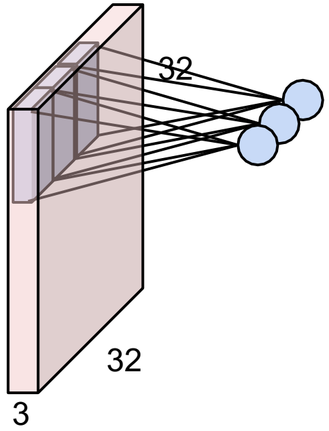
Convolution Layer



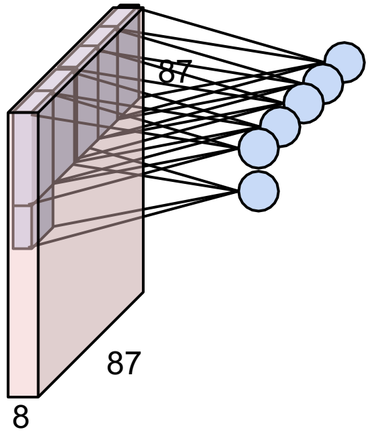
卷积层



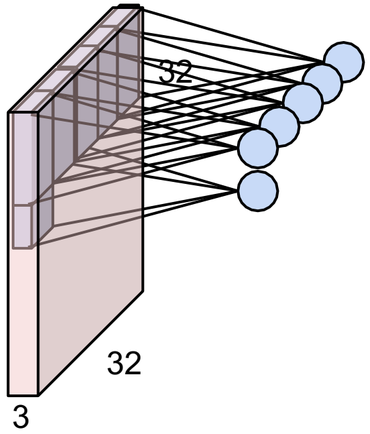
Convolution Layer



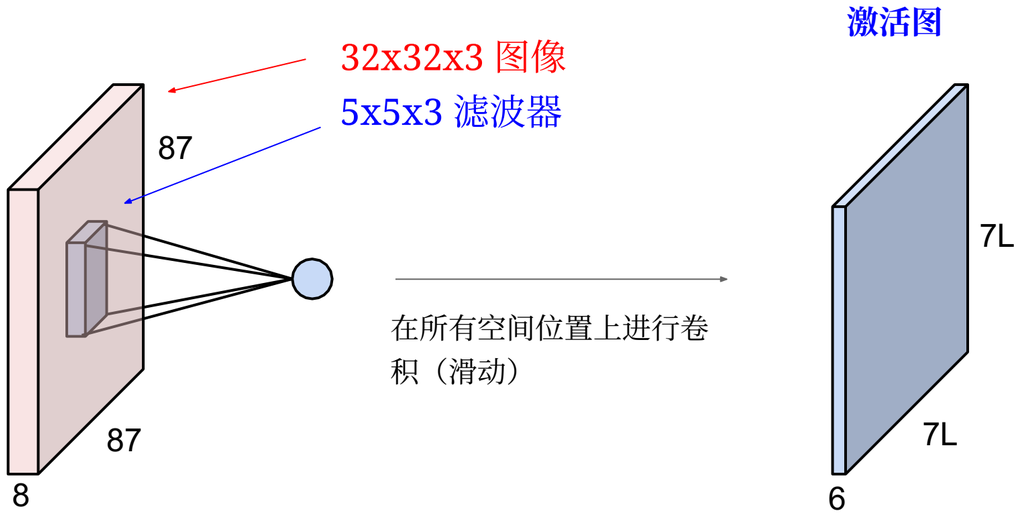
卷积层



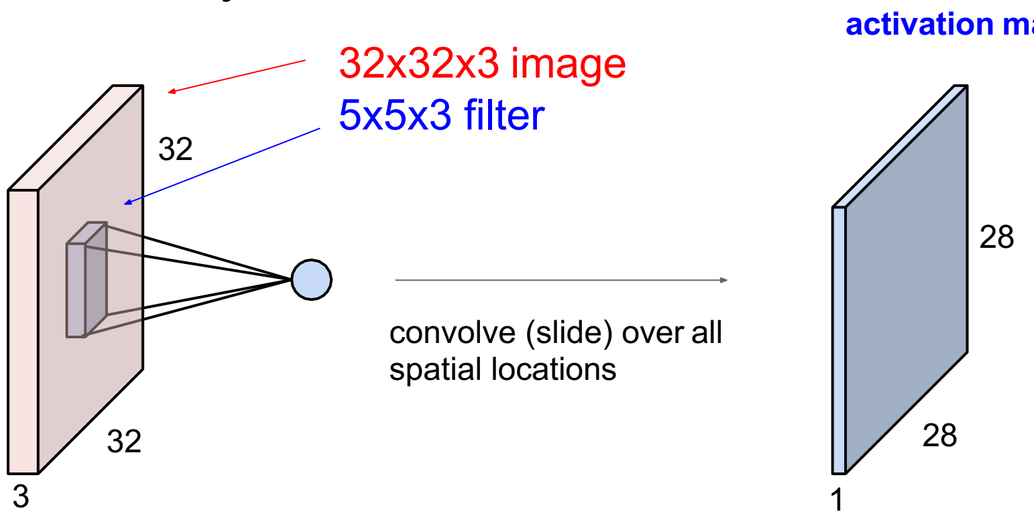
Convolution Layer



卷积层

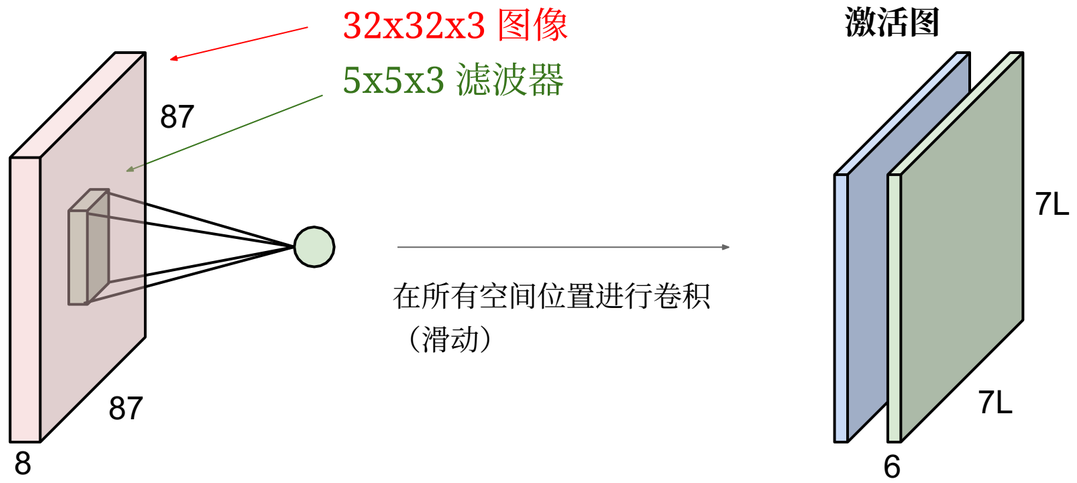


Convolution Layer



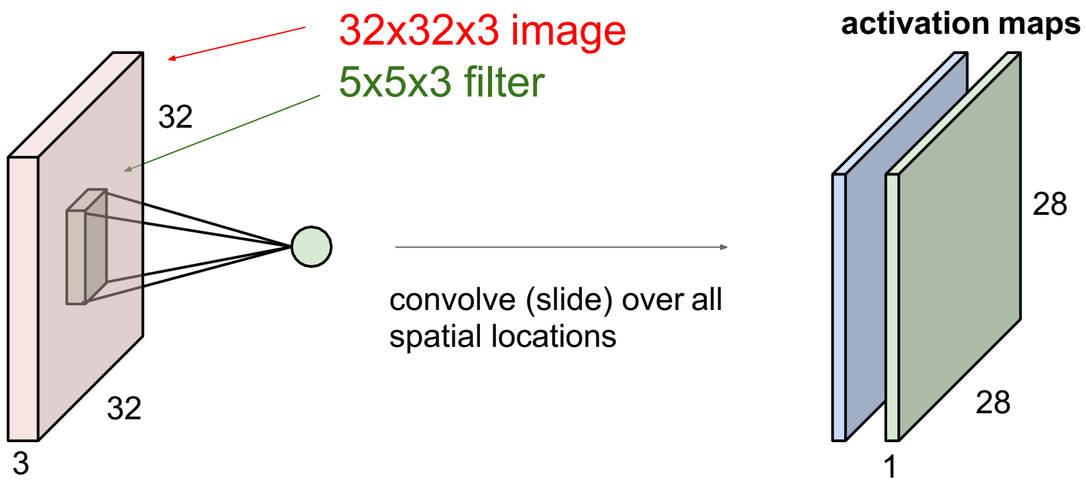
卷积层

考虑第二个绿色滤波器

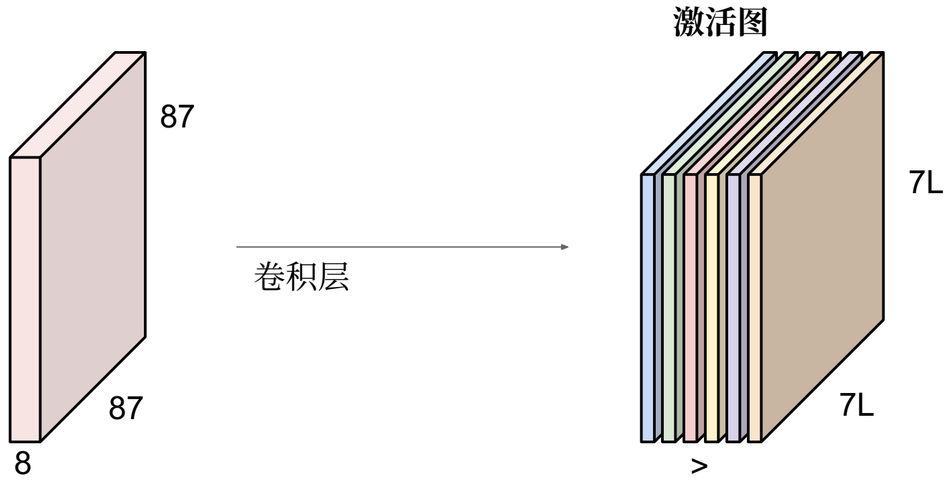


Convolution Layer

consider a second, green filter

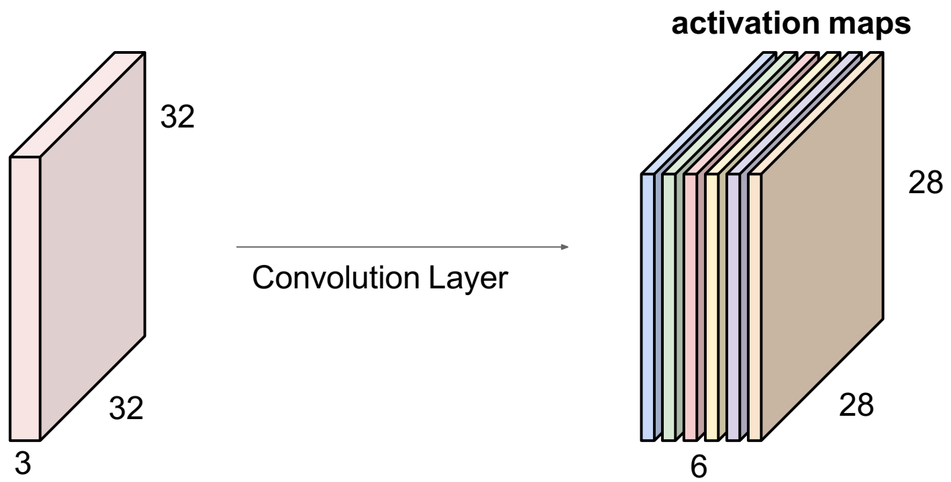


例如，如果我们有 6 个 5×5 滤波器，我们将得到 6 个单独的激活图：



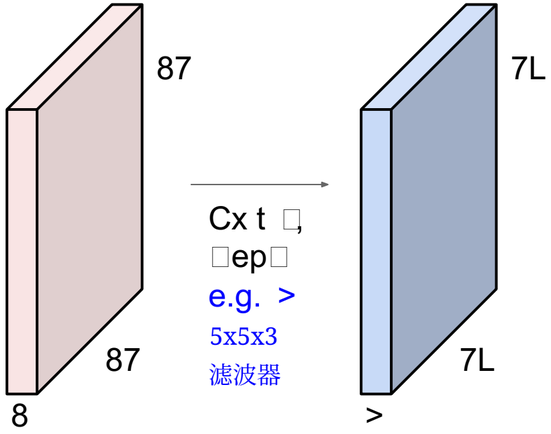
w 我们将这些堆叠起来，得到一张大小为 28x28x6 的“新图像”！

For example, if we had 6 5x5 filters, we'll get 6 separate activation maps:

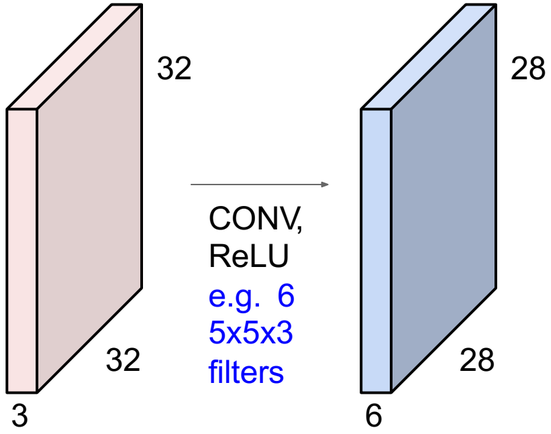


We stack these up to get a “new image” of size 28x28x6!

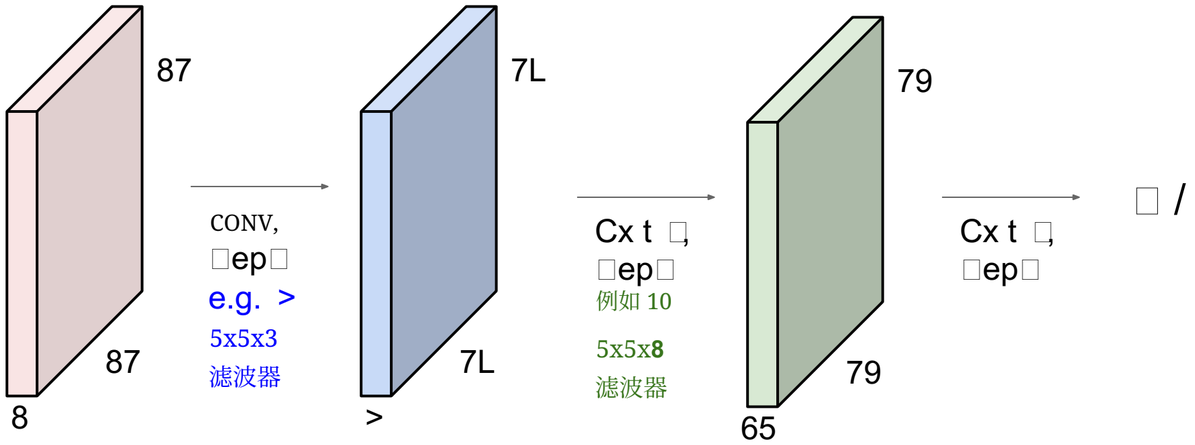
预览：卷积网络是一系列卷积层，其间穿插着激活函数



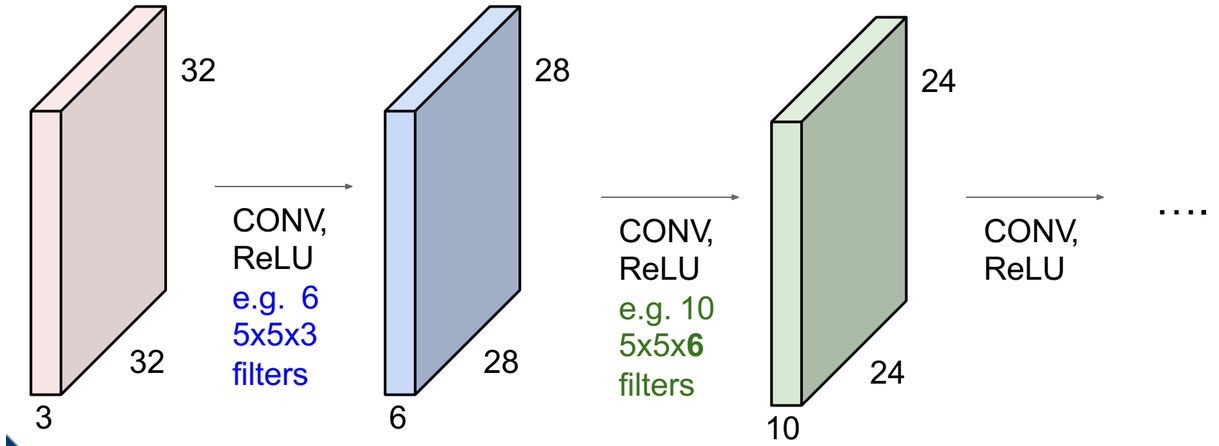
Preview: ConvNet is a sequence of Convolution Layers, interspersed with activation functions



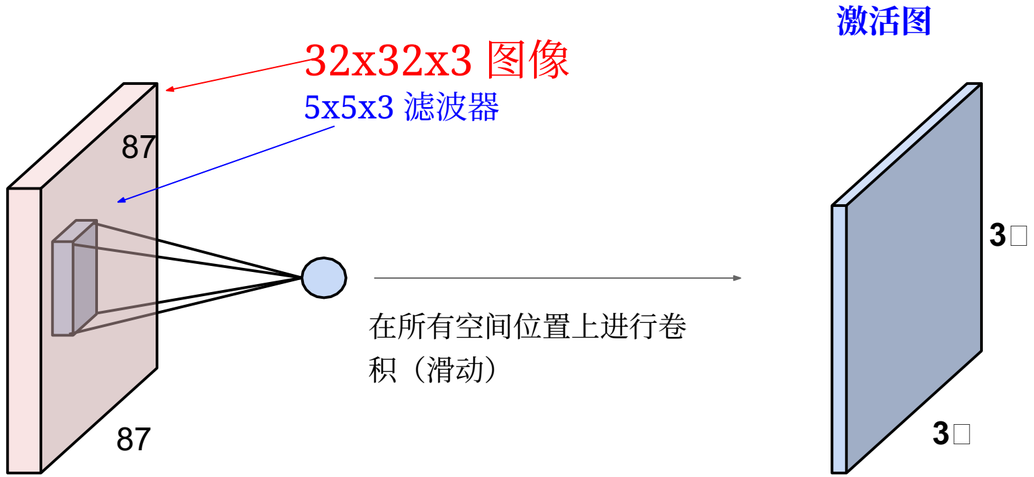
ConvNet 是一系列卷积层，其间穿插着激活函数



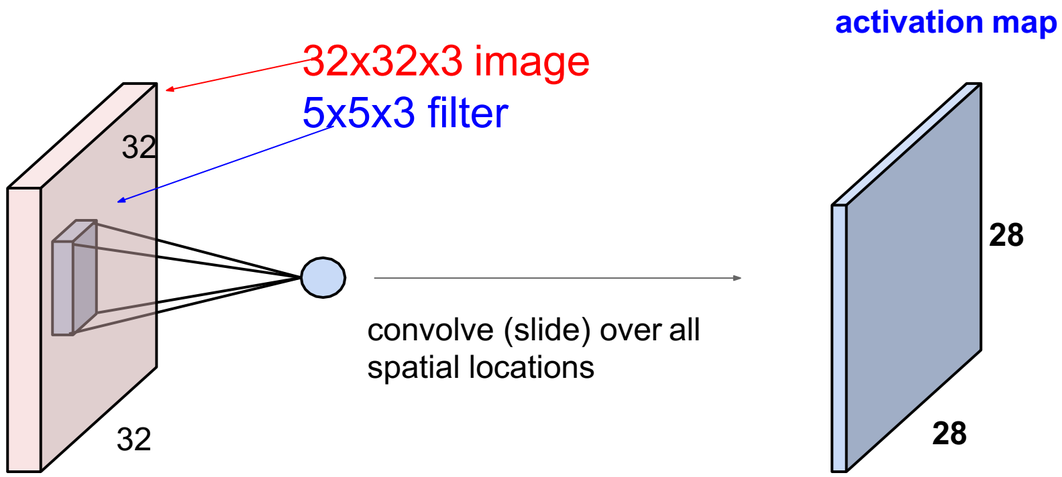
ConvNet is a sequence of Convolution Layers, interspersed with activation functions



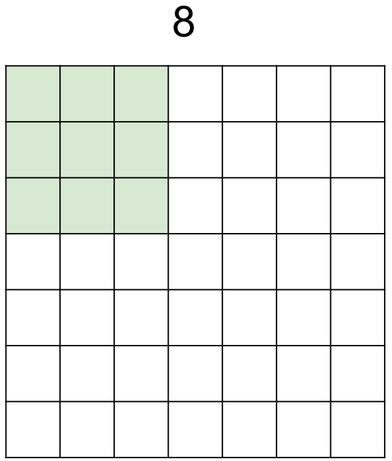
更深入地审视空间维度：



A closer look at spatial dimensions:

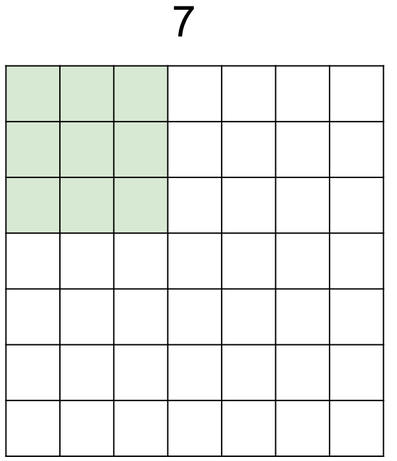


更深入地了解空间维度：



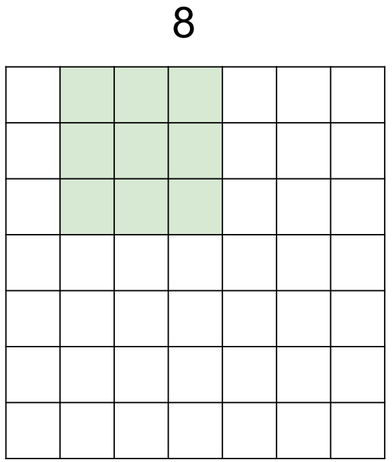
7x7 输入（空间上）假设
3x3 滤波器

A closer look at spatial dimensions:



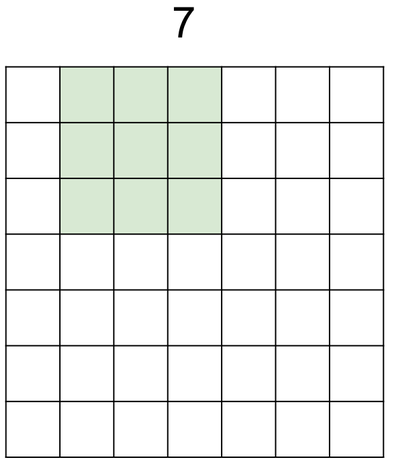
7x7 input (spatially)
assume 3x3 filter

深入了解空间维度：



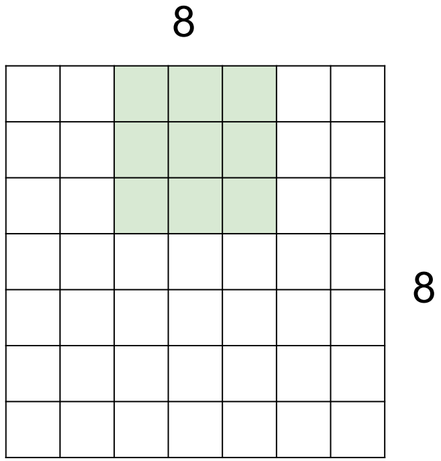
7x7 输入（空间上）假设
3x3 滤波器

A closer look at spatial dimensions:



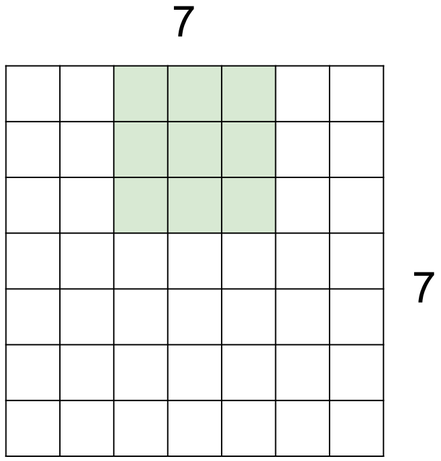
7x7 input (spatially)
assume 3x3 filter

更深入地观察空间维度：



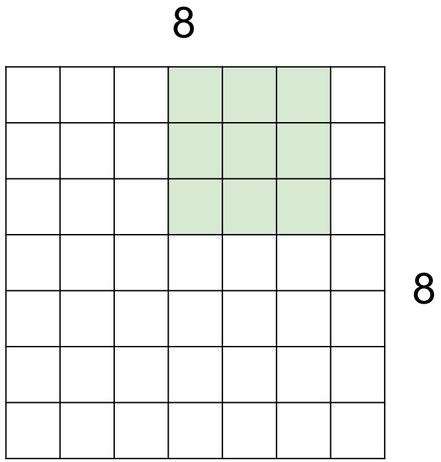
7x7 输入（空间上）假设
3x3 滤波器

A closer look at spatial dimensions:



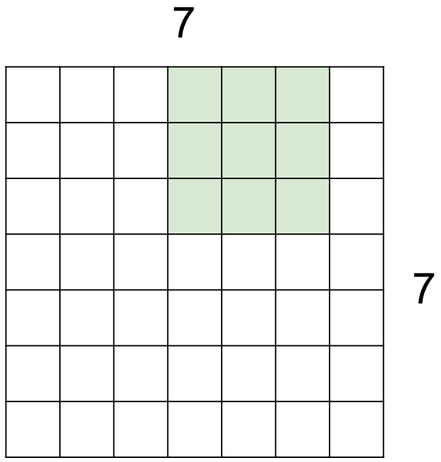
7x7 input (spatially)
assume 3x3 filter

深入了解空间维度：



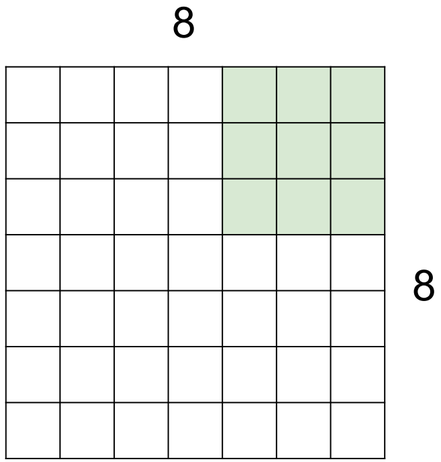
7x7 输入（空间上）假设
3x3 滤波器

A closer look at spatial dimensions:



7x7 input (spatially)
assume 3x3 filter

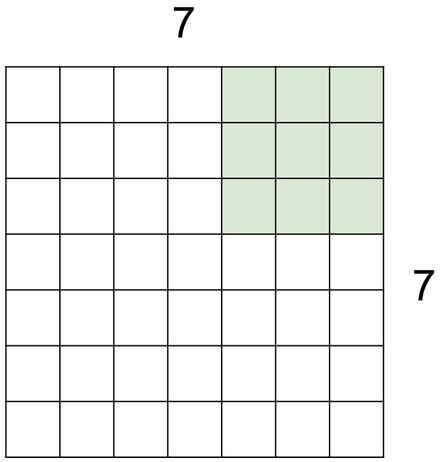
深入探讨空间维度：



7x7 输入（空间上）假设
3x3 滤波器

hn ex5 输出

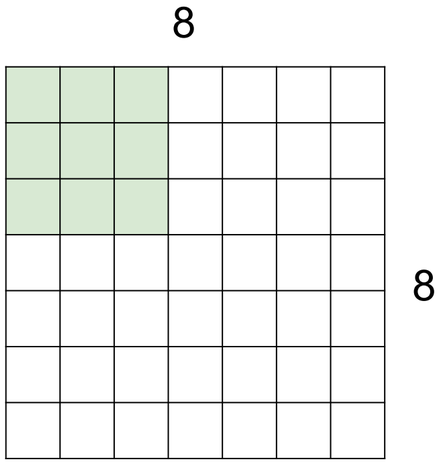
A closer look at spatial dimensions:



7x7 input (spatially)
assume 3x3 filter

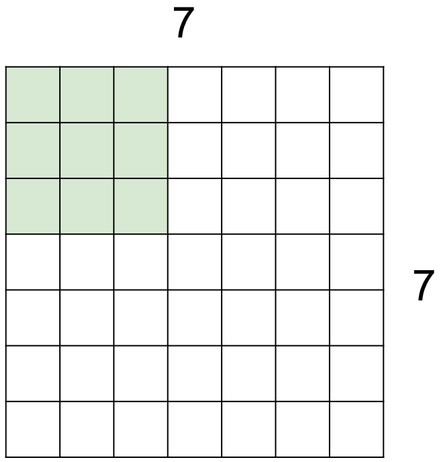
=> 5x5 output

深入了解空间维度：



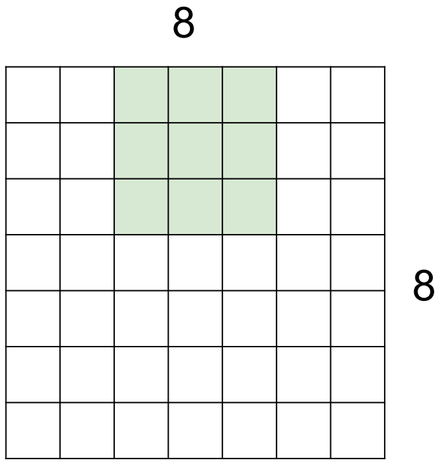
7x7 输入（空间上）假设应用 3x3 滤波器，**步幅为 2**

A closer look at spatial dimensions:



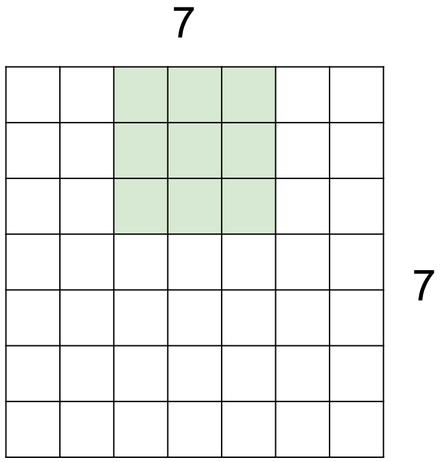
7x7 input (spatially) assume 3x3 filter applied **with stride 2**

深入了解空间维度：



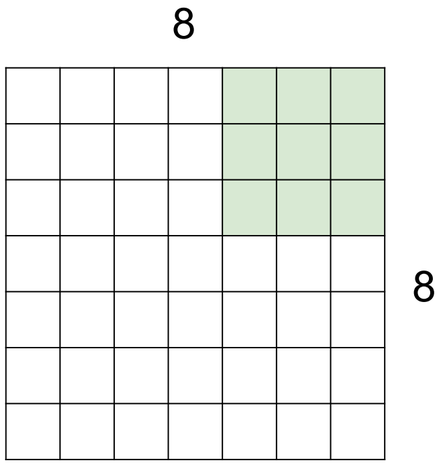
7x7 输入（空间上）假设应用 3x3 滤波器，**步幅为 2**

A closer look at spatial dimensions:



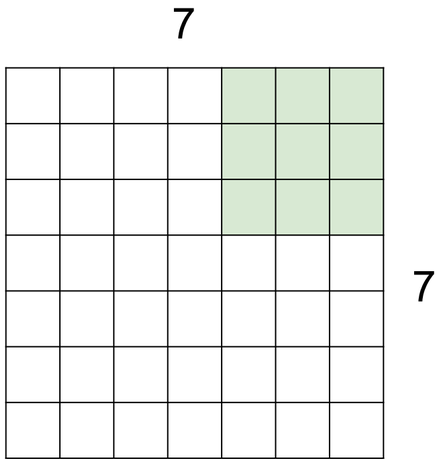
7x7 input (spatially) assume 3x3 filter applied **with stride 2**

深入了解空间维度：



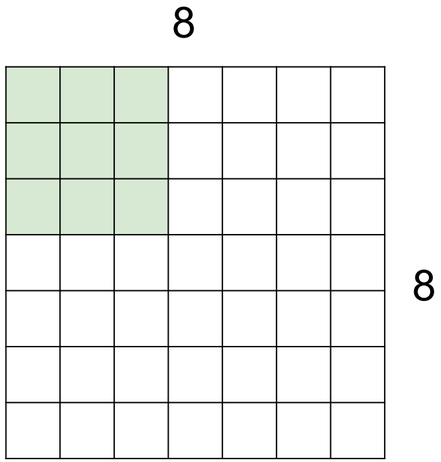
7x7 输入（空间上）假设应
用 3x3 滤波器 **使用步幅**
hn a a x 输出！

A closer look at spatial dimensions:



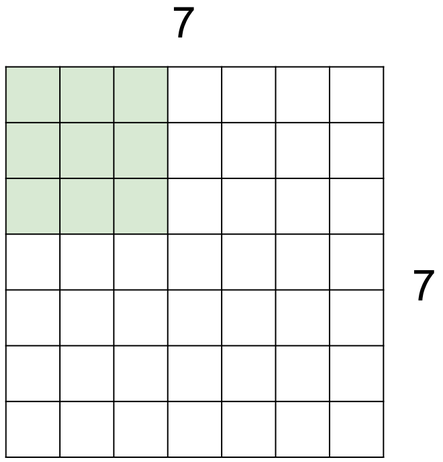
7x7 input (spatially)
assume 3x3 filter
applied **with stride 2**
=> 3x3 output!

深入了解空间维度：



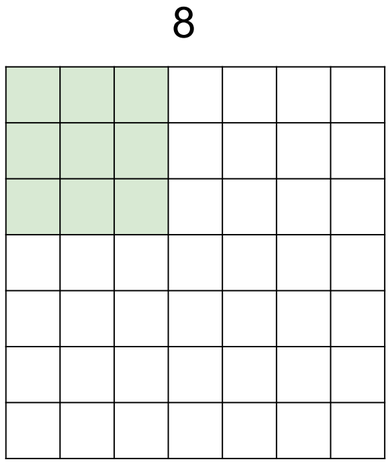
7x7 输入（空间上）假设应用 3x3 滤波器 **使用步幅 3?**

A closer look at spatial dimensions:



7x7 input (spatially) assume 3x3 filter applied **with stride 3?**

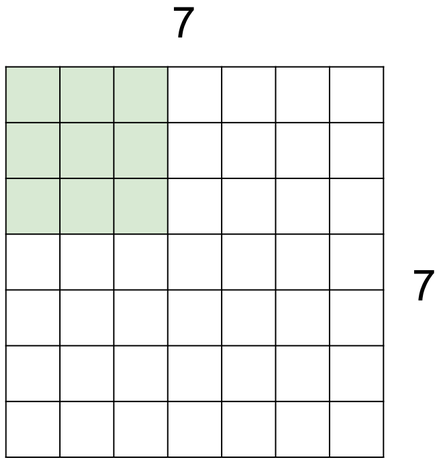
深入了解空间维度：



7x7 输入（空间上）假设应用 3x3 滤波器 **步幅为 3?**

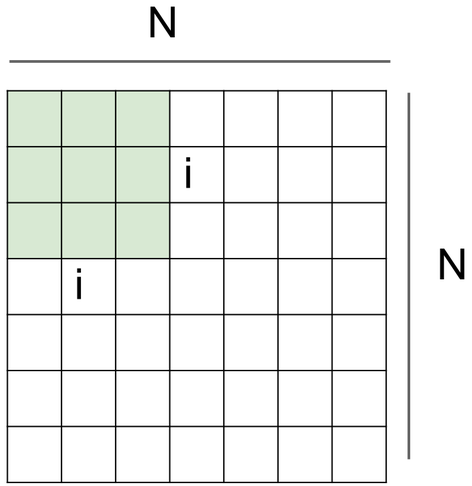
不匹配！无法以步长 3 在 7x7 输入上应用 3x3 滤波器。

A closer look at spatial dimensions:



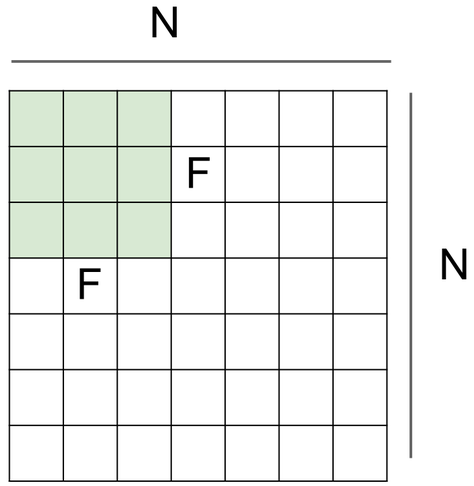
7x7 input (spatially) assume 3x3 filter applied **with stride 3?**

doesn't fit!
cannot apply 3x3 filter on 7x7 input with stride 3.



输出尺寸：**(N -**
F) / stride 3 R

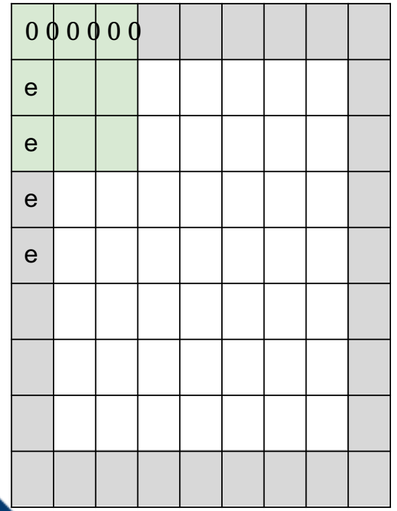
例如 N ? 8， F ? 4：步长 1 ?F (7 - 3)/1 ， 1 ? 6 步长 3 ?F (7 - 3)/ 3 ， 1 ? 4 步长 4 ?F (7 - 3)/4 ， 1 ? 3. 33



Output size:
(N - F) / stride + 1

e.g. N = 7, F = 3:
stride 1 => (7 - 3)/1 + 1 = 5
stride 2 => (7 - 3)/2 + 1 = 3
stride 3 => (7 - 3)/3 + 1 = 2.33

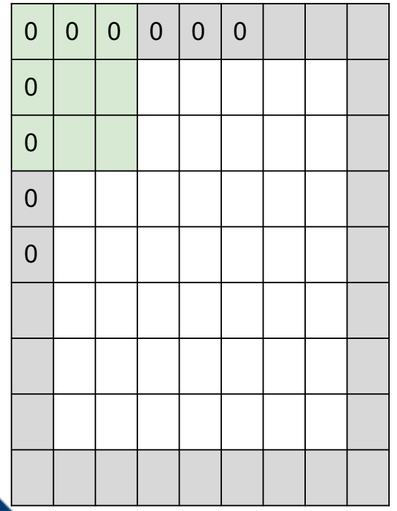
实践中：通常对边界进行零填充



例如 输入 7 7 □□x□滤波器，应用于 **步长 1 填充 1 像素** 边界 de 输出是什么?

（回顾：） (N - i) 0stri□e ， 1

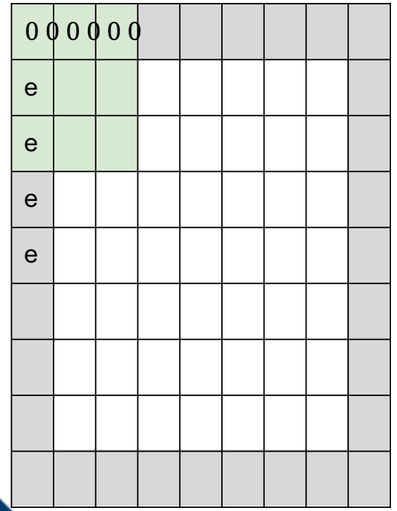
In practice: Common to zero pad the border



e.g. input 7x7
3x3 filter, applied with **stride 1**
pad with 1 pixel border => what is the output?

(recall:)
(N - F) / stride + 1

在实践中：通常对边界进行零填充

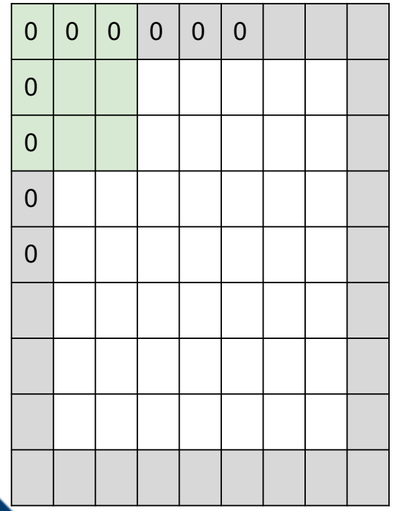


例如，输入 7 7 □□x□滤波器，应用时使用 **步幅 1，用 1 像素填充** 边界 de 输出是什么?

7x7 输出!

（回顾：） (N ， 3□ - i) 0stri□e ， 1

In practice: Common to zero pad the border

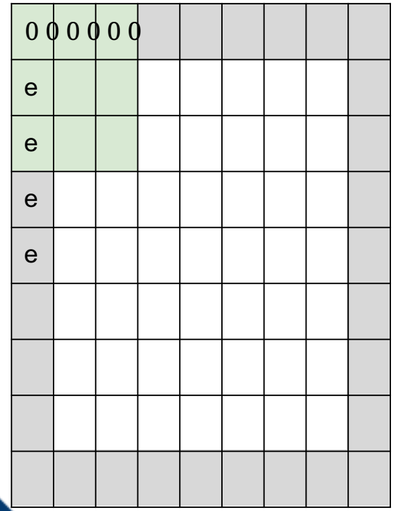


e.g. input 7x7
3x3 filter, applied with **stride 1**
pad with 1 pixel border => what is the output?

7x7 output!

(recall:)
(N + 2P - F) / stride + 1

实践中：通常对边界进行零填充

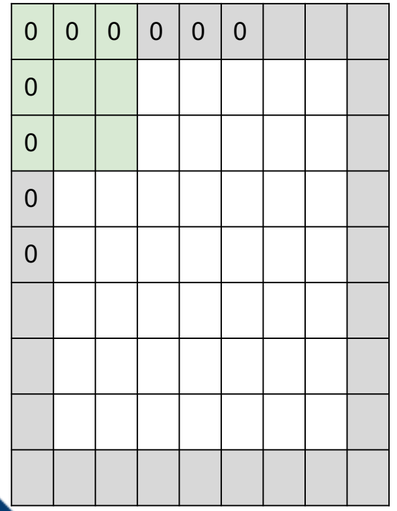


例如，输入 7 7 □□x□滤波器，应用时采用 **步幅 1，填充 1 像素** 边界 de 输出是什么?

7x7 输出!
通常，常见的是步幅为 1、滤波器大小为 FxF、零填充为 (F-1)/2 的卷积（CONV）层。（将在空间上保持尺寸）

e.g. i d > de zero pad □ith 1
i d F de zero pad □ith 2
i d [de zero pad □ith >

In practice: Common to zero pad the border

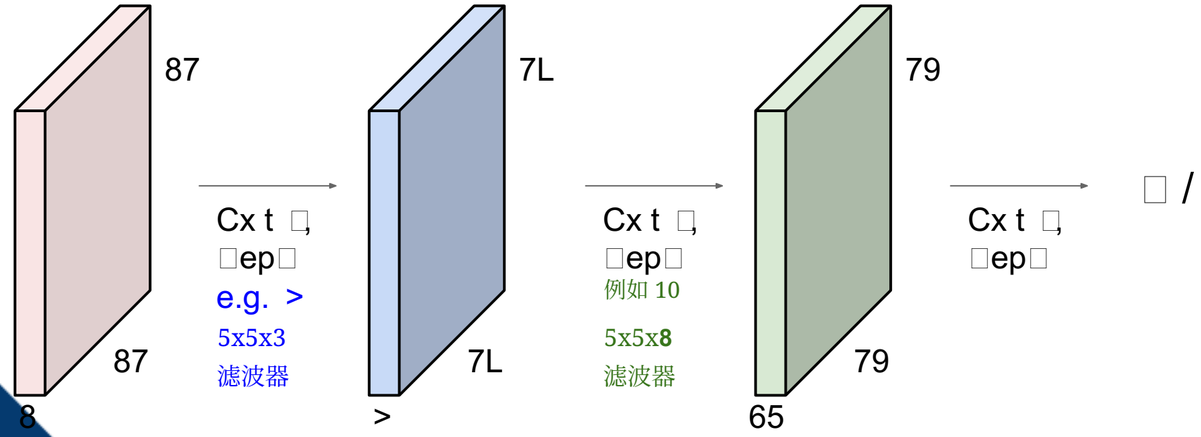


e.g. input 7x7
3x3 filter, applied with **stride 1**
pad with 1 pixel border => what is the output?

7x7 output!
in general, common to see CONV layers with stride 1, filters of size FxF, and zero-padding with (F-1)/2. (will preserve size spatially)
e.g. F = 3 => zero pad with 1
F = 5 => zero pad with 2
F = 7 => zero pad with 3

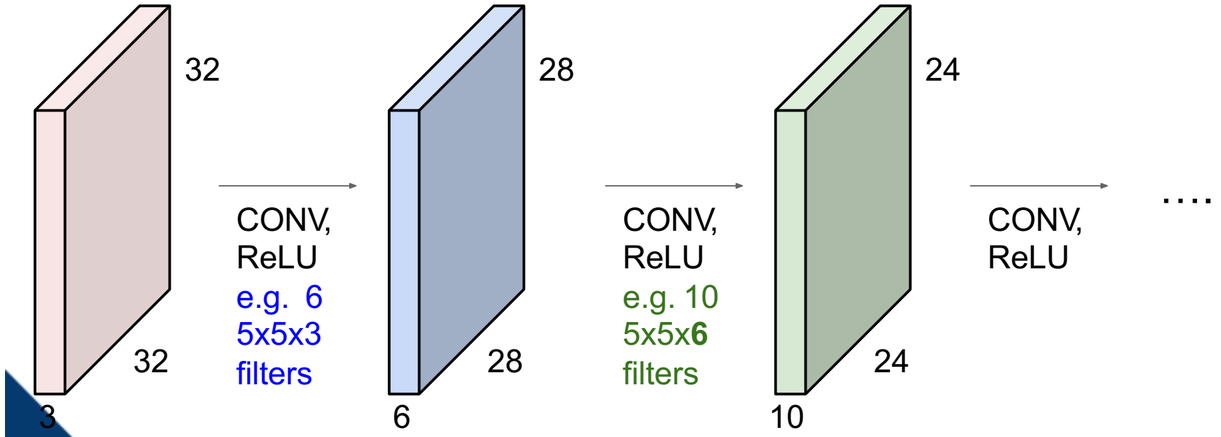
回想一下……

例如，32x32 的输入反复与 5x5 滤波器做卷积会使体积在空间上缩小！（32 -U 7L -U 79 ...）。压缩得太快并不好，效果也不佳。



Remember back to…

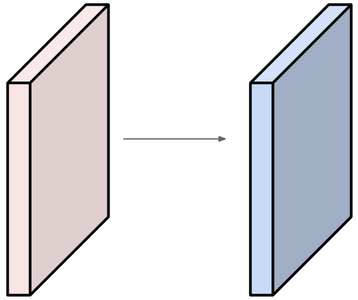
E.g. 32x32 input convolved repeatedly with 5x5 filters shrinks volumes spatially! (32 -> 28 -> 24 ...). Shrinking too fast is not good, doesn't work well.



示例时间：

输入体积：**323** 10 5**32**5 **xx**x 个滤波器，步幅为 1，填充为 2

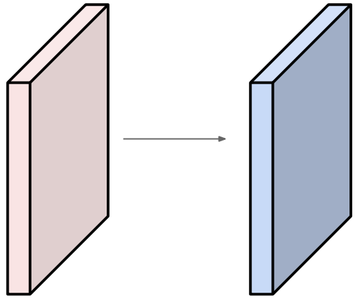
输出体积大小：？



Examples time:

Input volume: **32x32x3**
10 5x5 filters with stride 1, pad 2

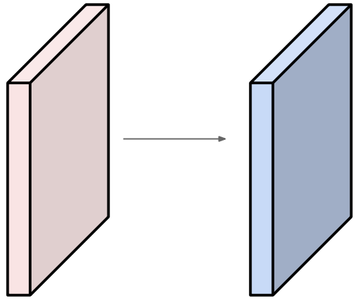
Output volume size: ?



示例时间：

输入体积：**323** 53 =**32**= **xx**□滤波器，步长为 5，填充6

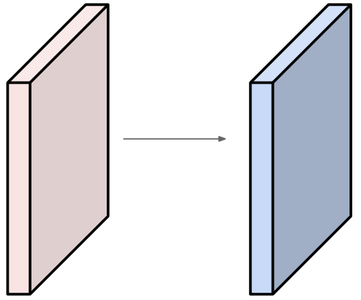
输出体积大小：（76- 6+6-=*25- 5 M76
在空间上，因此**32x32x10**



Examples time:

Input volume: **32x32x3**
10 5x5 filters with stride 1, pad 2

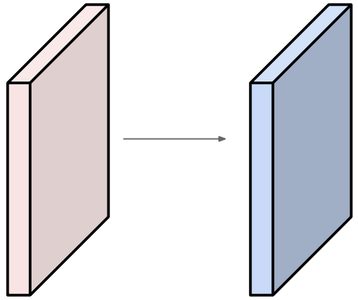
Output volume size:
(32+2*2-5)/1+1 = 32 spatially, so
32x32x10



示例时间：

输入体积：**323** 10 5**32**5 **xx**x 个滤波器，步长为 1，填充为 2

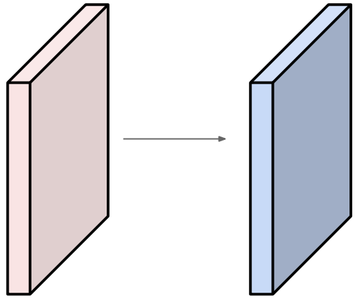
这一层有多少个参数?



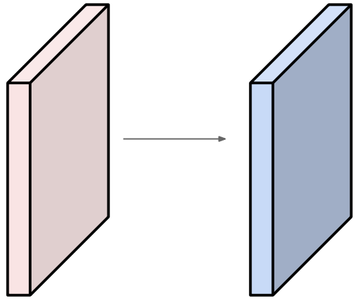
Examples time:

Input volume: **32x32x3**
10 5x5 filters with stride 1, pad 2

Number of parameters in this layer?



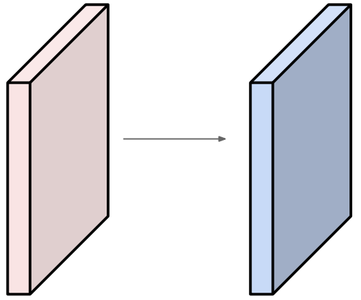
示例时间：



输入体积：**32x32x3**
53 =□= 步幅为 1、填充为 2 的滤波器

该层的参数数量是多少？
每个滤波器有 =+=+7 - 5 ME> 参数 M> E>+53 M**760** (. 1 用于偏置)

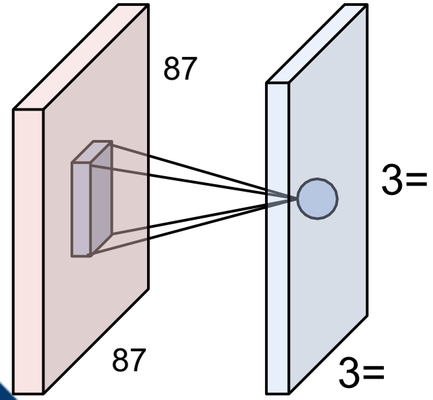
Examples time:



Input volume: **32x32x3**
10 5x5 filters with stride 1, pad 2

Number of parameters in this layer?
each filter has 5*5*3 + 1 = 76 params (+1 for bias)
=> 76*10 = **760**

感受野



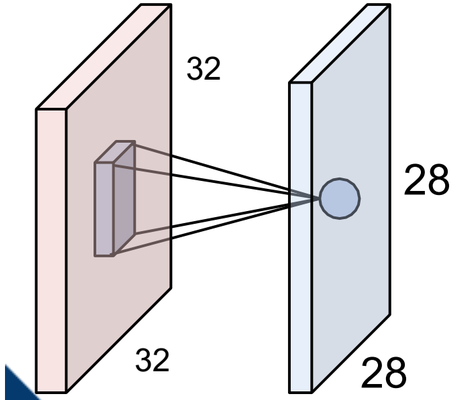
激活图是一张 28x28 的神经元输出表：

1. 每个都连接到输入中的一个小区域 2. 它们全部共享参数

“5x5 滤波器” -U“每个神经元的 5x5 感受野”

https://distill.pub/2019/computing-receptive-fields/

Receptive field



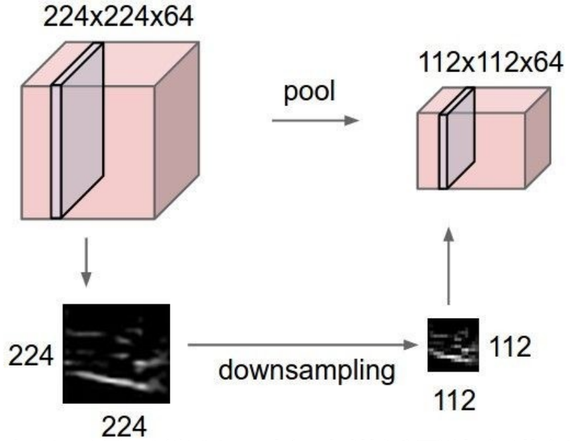
An activation map is a 28x28 sheet of neuron outputs:

1. Each is connected to a small region in the input
2. All of them share parameters

“5x5 filter” -> “5x5 receptive field for each neuron”

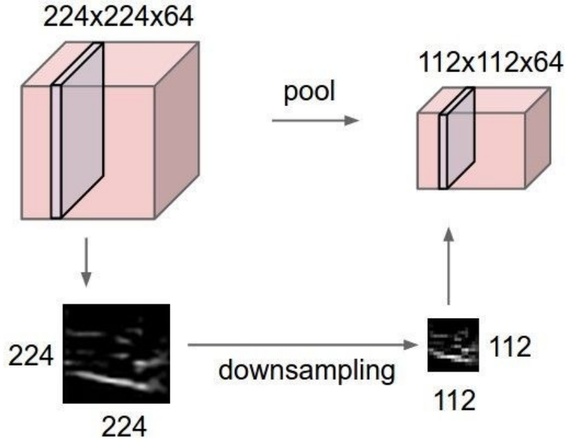
https://distill.pub/2019/computing-receptive-fields/

池化层



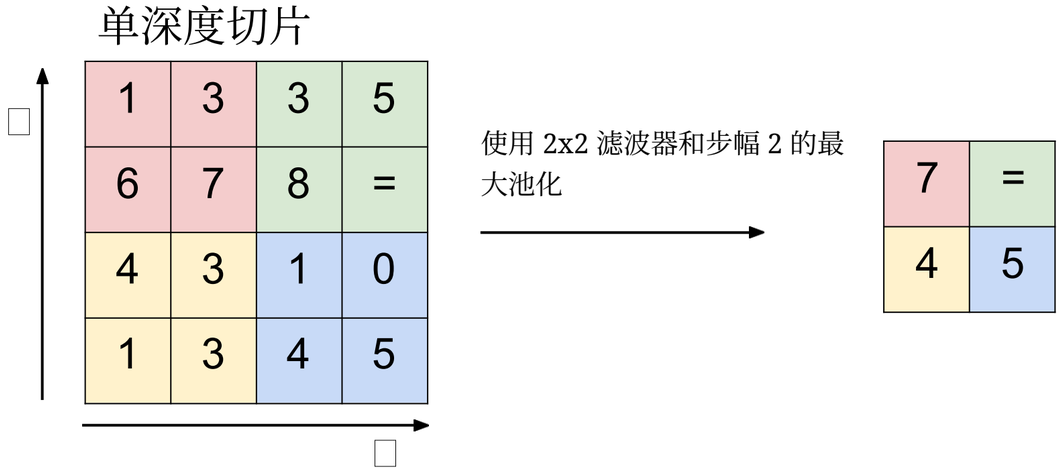
- 使表示更小且更易于管理 - 对每个激活图独立进行操作:

Pooling layer

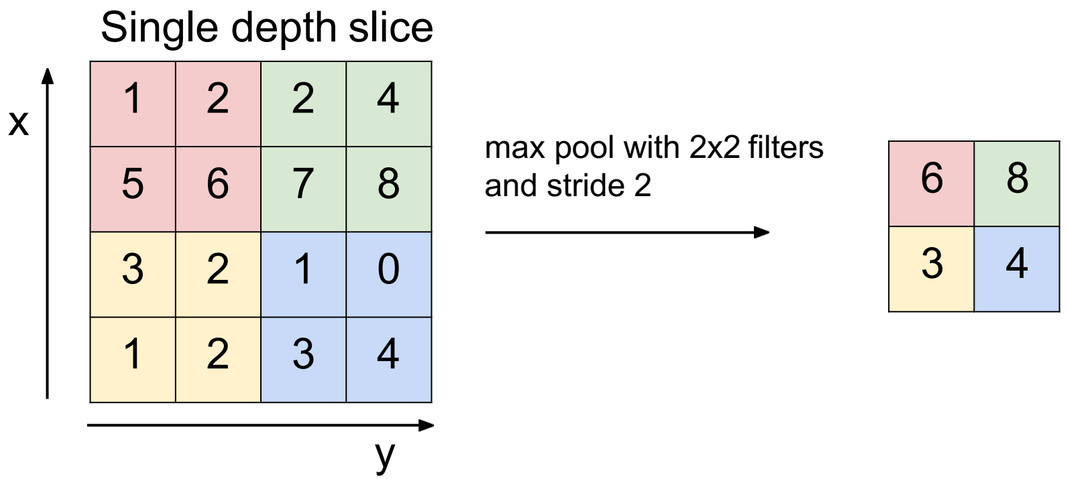


- makes the representations smaller and more manageable
- operates over each activation map independently:

最大池化



Max pooling



匿名课堂反馈

Q&A

Anonymous
Class Feedback

Q&A